



Universidade de Brasília - UnB
Faculdade UnB Gama - FGA
Engenharia Eletrônica

Deteccção de Movimento por Imagens Segmentadas Utilizando Raspberry

Autor: Rick Eichi Arnor Yamamoto
Orientador: Prof. Dr. Gerardo Antonio Idrobo Pizo

Brasília, DF
2022



Rick Eichi Arnor Yamamoto

Detecção de Movimento por Imagens Segmentadas Utilizando Raspberry

Monografia submetida ao curso de graduação em (Engenharia Eletrônica) da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia Eletrônica.

Universidade de Brasília - UnB

Faculdade UnB Gama - FGA

Orientador: Prof. Dr. Gerardo Antonio Idrobo Pizo

Brasília, DF

2022

Rick Eichi Arnor Yamamoto

Detecção de Movimento por Imagens Segmentadas Utilizando Raspberry/ Rick
Eichi Arnor Yamamoto. – Brasília, DF, 2022-

73 p. : il. (algumas color.) ; 30 cm.

Orientador: Prof. Dr. Gerardo Antonio Idrobo Pizo

Trabalho de Conclusão de Curso – Universidade de Brasília - UnB
Faculdade UnB Gama - FGA , 2022.

1. Segmentação de Imagens. 2. Raspberry Pi 3. I. Prof. Dr. Gerardo Antonio
Idrobo Pizo. II. Universidade de Brasília. III. Faculdade UnB Gama. IV. Detecção
de Movimento por Imagens Segmentadas Utilizando Raspberry

CDU 02:141:005.6

Rick Eichi Arnor Yamamoto

Deteccção de Movimento por Imagens Segmentadas Utilizando Raspberry

Monografia submetida ao curso de graduação em (Engenharia Eletrônica) da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia Eletrônica.

Trabalho aprovado. Brasília, DF, 28 de setembro de 2022:

**Prof. Dr. Gerardo Antonio Idrobo
Pizo**
Orientador

**Prof. Dr. José Maurício Santos Torres
da Motta (ENM/FT/UnB)**
Convidado 1

**Prof. Dr. Daniel Mauricio Muñoz
Arboleda(FGA/UnB)**
Convidado 2

Brasília, DF
2022

Agradecimentos

Agradeço primeiramente a minha família que me apoiou desde o início da escolha desse curso até este prezado momento, apesar das dificuldades as quais me deparei na graduação sempre me assistiram.

Ao excelentíssimo professor-orientador o qual me deu suporte e conhecimento suficiente para que me permitisse finalizar e completar mais uma etapa para me conceder o título de Bacharel.

Aos meus queridos amigos colegas de graduação que eu tive o prazer de conhecê-los durante essa fase da vida de extrema importância onde, logo imaturos, temos que decidir uma das escolhas mais importante de nossas vidas, agradeço-lhes o suporte os quais têm me proporcionado.

Resumo

Com alta demanda de informações disponíveis, é necessário filtrar essas informações mais rentável possível. Para tanto, o dispositivo Raspberry Pi 3 será utilizado para captação de objetos ou pessoas com a assistência de uma câmera a captação de imagem em tempo real para análise de imagens segmentadas pelo método de Otsu e armazenada apenas objetos em movimento, de forma a otimizar o tempo de análise qualitativa do usuário. Implementa-se a captação de imagem diretamente da Raspberry ou via *streaming*, utilizando a interface Open CV e redução de ruídos na presença de tons de cinza aplicáveis e processado em ambientações adversas e variáveis. Os resultados obtidos indicam que as melhores condições de eficiência na média de *F-measure* de 88,4% estão em ambiente com iluminação estável e numa intensidade de luminosidade de 230 lumens, posicionamento da câmera inerte e alto contraste de objeto e fundo, com uma taxa de 10 FPS e 1 FPS via *streaming*. O projeto teve os objetivos alcançados, visto que a eficiência de armazenamento foi de 92,6% e o *F-measure* maior que 90% em luminosidade estável e visível, por utilizar apenas um microcomputador e um periférico, o orçamento do projeto se torna de baixo custo de produção.

Palavras-chaves: Segmentação de Imagens, Detecção de movimento, Processamento de Imagens, Raspberry Pi 3, *Streaming*, Método de Otsu, OpenCV, Python.

Abstract

With the high demand of available data, it is necessary to filter those informations in the most rentable way. To do so, a Raspberry Pi 3 device, assisted by a camera, is used to take pictures of the environment in real time. By segmenting the images using Otsu's method, the system is able to detect the presence of people and objects, and also save only frames that have movement, in order to optimize the user qualitative analysis time. The image capturing process is implemented directly from the Raspberry camera or via *streaming*, with usage of the OpenCV library and noise reduction techniques applied to grayscale images in adversarial and variable environments. The best results point out that the best efficiency conditions have an average *F-measure* of 88.4% in environments with stable lighting and light intensity of 230 lumens, inert camera positioning and high contrast between the object and the background, with a rate of 10 FPS, by using the camera directly, and 1 FPS via *streaming*. The project was able to achieve its goals, since the storage efficiency was 92,6% and the *F-measure* was greater than 90% in environments with visible and stable luminosity. Since only a microcomputer and a peripheral device is used, the project is considered to be a low-cost solution for the problem of image segmentation in real time.

Key-words: Image Segmentation, Motion detection, Image Processing, Raspberry Pi 3, Method of Otsu, OpenCV, Streaming, Python.

Lista de ilustrações

Figura 1 – Diagrama representando o fluxo de processamento das imagens	22
Figura 2 – Fluxograma PRISMA	24
Figura 3 – Diagrama de processamento de imagens	26
Figura 4 – Visão esquemática da câmera CCD	27
Figura 5 – Imagem policromática RGB	28
Figura 6 – Imagem em tons de cinza	30
Figura 7 – Imagem binarizada	30
Figura 8 – Representação de um histograma	31
Figura 9 – Exemplos de otimização de histograma	31
Figura 10 – Forma da distribuição Gaussiana em 2D média (0,0) e desvio padrão 1	34
Figura 11 – Imagem fornecida por diferenças de frames captadas	35
Figura 12 – Verdadeiro positivo	37
Figura 13 – Verdadeiro negativo	37
Figura 14 – Falso positivo	38
Figura 15 – Falso negativo	38
Figura 16 – Raspberry Pi 3 Model B	39
Figura 17 – Câmera Module V2 OmniVision OV5647	40
Figura 18 – Esquemático comunicação da câmera	40
Figura 19 – Lâmpada inteligente Elgin	41
Figura 20 – Fluxo da codificação MPEG	42
Figura 21 – Raspberry Pi 3 e Câmera	43
Figura 22 – Imagem captada colorida	44
Figura 23 – Imagem captada segmentada	44
Figura 24 – Interface do navegador com a captação de imagem via <i>streaming</i>	45
Figura 25 – Imagem captada em escala de cinza	45
Figura 26 – Histograma gerado pelo algoritmo	46
Figura 27 – Diagrama de máquina de estado da detecção de movimento	47
Figura 28 – Imagem captada objeto em movimento	48
Figura 29 – Imagem captada objeto em movimento segmentada	48
Figura 30 – Imagem captada objeto em movimento via <i>streaming</i>	50
Figura 31 – Imagem coletada de Taguatinga Shopping	50
Figura 32 – Imagem coletada de rua período noturno	51
Figura 33 – Imagem captada em 4% em movimento em relação à imagem total . .	52
Figura 34 – Esquemático do experimento movimento pendular	52
Figura 35 – Imagem captada em 1% de intensidade luminosa	53
Figura 36 – Imagem captada em 33% de intensidade luminosa	53

Figura 37 – Imagem captada em 66% de intensidade luminosa	54
Figura 38 – Imagem captada em 100% de intensidade luminosa	54
Figura 39 – Exemplo de imagem policromática captada com fundo	55
Figura 40 – Exemplo de imagem segmentada captada com fundo	55
Figura 41 – Exemplo de imagem em níveis de tons de cinza	56
Figura 42 – Comparativo de histograma extraído do programa ImageJ	56

Lista de tabelas

Tabela 1	–	Tipos de Imagens (SOUSA, 2019)	26
Tabela 2	–	Tabela de especificações da Raspberry Pi 3 Model B	39
Tabela 3	–	Especificações da lâmpada Elgin	41
Tabela 4	–	Módulos e funções da biblioteca OpenCV	42
Tabela 5	–	Resultados coletados de locais observados	50
Tabela 6	–	Resultados coletados de luminosidade variável	53

Lista de abreviaturas e siglas

OpenCV	<i>Open Source Computer Vision</i>
V2	Versão Dois
3D	Três dimensões
2D	Duas dimensões
CMOS	Semicondutor de óxido metálico complementar
SCV	Sinal Composto de Vídeo
RGB	<i>Red, Green, Blue</i>
CCD	<i>Charge-coupled device</i>
JPEG	<i>Joint Photographic Experts Group</i>
MPEG	<i>Moving Picture Experts Group</i>
AVI	<i>Audio Video Interleaved</i>
PES	<i>Packetized Elementary Stream</i>
PC	Computador Pessoal
OpenGL	<i>Open Graphics Library</i>
HD	Alta Definição
I2C	<i>Inter-Integrated Circuit</i>
CSI	<i>Camera Serial Interface</i>
UI	Interface de usuário
FPS	Frames por segundo
TCC	Trabalho de conclusão de curso

Lista de símbolos

$f(x, y)$	Função de intensidade luminosa
$g(x, y)$	Função de binarização
R_1	Valor de referência ao pixel preto
R_2	Valor de referência ao pixel branco
T	Valor do <i>Thresholding</i> ideal
C_0	Tonalidade de cinza da classe 0
C_1	Tonalidade de cinza da classe 1
t	Ponto de corte da limiarização
σ_w^2	Variância dentro da classe
σ_b^2	Variância entre as classes
σ_t^2	Variância total
λ, K, N	Variáveis de referência de limiarização
P_i	probabilidade de ocorrência do nível de cinza i
n_i	Número de <i>pixels</i> como o nível de cinza i
ω_0, ω_1	Variáveis auxiliares
μ_0	Média de valores da classe 0
μ_1	Média de valores da classe 1
μ_t	Média total das classes

Sumário

1	INTRODUÇÃO	21
1.1	Contextualização	21
1.2	Definição do problema	21
1.3	Objetivo	22
1.3.1	Objetivo Geral	22
1.3.2	Objetivo específico	23
1.4	Fluxograma PRISMA	23
1.5	Organização do trabalho	23
2	FUNDAMENTAÇÃO TEÓRICA	25
2.1	Processamento de imagens	25
2.1.1	Imagem digital bidimensional	25
2.1.2	Aquisição de imagem	27
2.1.3	Digitalização	27
2.1.4	Segmentação de imagens	28
2.1.5	Binarização	29
2.1.6	Histograma	29
2.1.7	Método de Otsu	31
2.1.8	Filtro <i>GaussianBlur</i>	34
2.2	Deteção de movimento	35
2.2.1	Subtração de frames	35
2.3	Métrica de Avaliação	36
2.3.1	<i>F-measure</i>	36
2.4	Hardware	38
2.4.1	Raspberry Pi 3 Model B	38
2.4.2	Especificações da Raspberry Pi 3 Model B	39
2.4.3	Camera Module Omnivision OV5647	39
2.4.4	Lâmpada LED Elgin	40
2.5	Software	41
2.5.1	OpenCV	41
2.6	Captação em <i>streaming</i>	42
3	METODOLOGIA	43
3.1	Algoritmo	43
3.1.1	Captação de imagem em tempo real	43
3.1.2	Cálculo do <i>Threshold</i> de Otsu	44

3.1.3	Cálculo do histograma	46
3.2	Gravação de objetos em movimento	46
3.2.1	Diagrama de máquina de estado	47
4	RESULTADOS	49
4.1	Captação da imagem em movimento	49
4.1.1	Captação de pequenos objetos	51
4.1.2	Captação em luminosidade variável	51
4.2	<i>Threshold</i> de Otsu	53
4.3	Histograma gerado	54
5	CONCLUSÃO	57
5.1	Trabalhos futuros	58
	REFERÊNCIAS	59
	APÊNDICES	61
	APÊNDICE A – APRESENTAÇÃO DO TCC	63

1 Introdução

1.1 Contextualização

Com a alta troca de informações digitais em servidores, uma forma de organizá-las é por segmentação das imagens armazenadas, captando apenas o objeto desejado. Devido a isso, pela enorme quantidade de dados armazenados, o presente projeto teve intuito de diminuir a superlotação de armazenamento desses servidores, por meio da implementação de um algoritmo de segregação de imagens, pauta levantada recentemente por [Moraes e Silva \(2021\)](#) na avaliação da sobrecarga de bancos e dados.

Nesse contexto, no que abrange a área dos autômatos, é possível afirmar que um fluxo de dados e de informação requer uma organização de processamento de informação capaz de otimizar os servidores, onde são armazenadas as informações disponíveis ([ELMASRI, 2008](#)). Um projeto de otimização de captação de imagens é capaz de auxiliar monitores de segurança a adquirir informações significativas de informações a serem processadas e analisadas, de forma a organizar os dados de forma simplificada e intuitiva.

Sendo assim, a Raspberry Pi 3 é um dispositivo que se apresenta como uma opção de captação de imagem transmitida via protocolo de comunicação serial, bastante utilizado para projetos de reconhecimento facial e análise de formas e contornos. O intuito da utilização da câmera permitirá recolher imagens movimentadas em tempo real, O qual dispositivo utilizado por [Pereira e Oliveira \(2016\)](#) na detecção de pessoas em imagens implementando técnicas de visão computacional em um Raspberry Pi.

Os processos serão separados em etapas onde será feito a captura de imagem em movimento, armazenar no formato PCX caracterizado por escalar em 256 tons de cinza, binarização da imagem para que possa ser preprocessada e eliminar possíveis ruídos com o filtro Gaussiano. Após esse processo é realizado a Segmentação dos objetos por *Background subtraction* na imagem com intuito de serem analisadas e armazenadas em um dispositivo de armazenamento.

1.2 Definição do problema

Com intuito de economizar espaço no dispositivo de armazenamento, o projeto tem como finalidade apenas captar imagens em movimento em espécie, para fins de processá-las e analisa-las de acordo com o interesse do usuário, otimizando o tempo de análise de informações adquiridas.

Ainda não há uma necessidade urgente para uma drástica mudança de paradigmas

no que se refere ao armazenamento de informações digitais, visto que a disponibilidade de espaço para se guardar estes dados é enorme em dispositivos menores e compactos. Entretanto, haverá uma necessidade futura do sistema tender a captar tanta informação que os servidores retardarão, assim como seus processos de armazenamento e consequentemente irão gerar uma sobrecarga.

Um dos requisitos desejados para este projeto foi a rentabilidade. Uma das preocupações foi o custo total do projeto, por isso os meios devidamente escolhidos foram os periféricos de baixo custo como uma Raspberry Pi 3 Model B e uma câmera integrada. Os periféricos atendem os requisitos para esse projeto, permitindo produzir em larga escala num custo acessível para quem deseja aderir ao projeto.

Tal projeto visa a otimização de desempenho dos serviços empregados para análise de imagens e informações em locais requisitados por filmagens de segurança e adaptação digital com intuito de aprimorar efeitos de captação e armazenamento de imagens em um local reduzido.

1.3 Objetivo

1.3.1 Objetivo Geral

O objetivo principal deste trabalho é captar e processar imagens coloridas em movimento para otimização de espaço e tempo de análise do mesmo. Tais imagens captadas serão convertidas em escalas de tons de cinza com a finalidade de ser segmentada pelo método de Otsu. Este projeto tem o objetivo similar apontado na tese de [Bruxel \(2015\)](#), de tal forma a trabalhar com a captação de imagens em movimento. Após o processamento de imagens é armazenada no diretório desejado.

Os algoritmos compreendem a tradução dos processos de imagens digitais, a partir deles, decidirão se a imagem tem conteúdo suficiente para ser armazenada em servidores locais ou virtuais a partir de uma câmera integrada da Raspberry com reconhecimento de movimento.

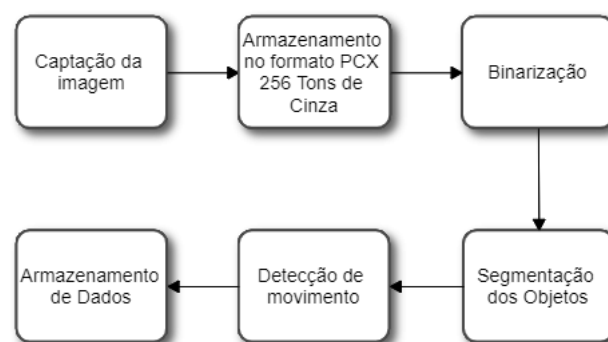


Figura 1 – Diagrama representando o fluxo de processamento das imagens

1.3.2 Objetivo específico

- Captação de imagens por protocolo de comunicação serial a partir da câmera integrada transmitida remotamente em tempo real;
- Utilização do Método de Otsu em algoritmo em Python para obter o histograma da imagem e realizar a segregação da imagem em duas tonalidades, branca e preta;
- Por meio de detecção de movimento, analisar o comportamento das imagens comparativamente em relação ao tempo;
- Reconhecer imagens de objetos/pessoas em movimento para serem processadas por algoritmos e armazenada na unidade de armazenamento;
- Captar imagens via *streaming* e processá-las para um comparativo a partir da captação direta na Raspberry;
- Testes em situações adversas para estabelecer a melhor e a pior condições de resultados;
- Processar a imagem coletada a partir da detecção de movimento com intuito de gerar um histograma e salvar no dispositivo;

1.4 Fluxograma PRISMA

O intuito de aplicar a revisão sistemática PRISMA é o auxílio de organização das revisões sistemáticas e meta-análise, a figura 2 demonstra a construção e modelo utilizado.

O fluxograma consiste em quatro etapas: Identificação, seleção, elegibilidade e inclusão. De forma a incluir estudos e decidir a exclusão de outros, onde resulta a busca de números de relatos encontrados, após essa análise, será decidido quais artigos os quais serão incluídos no texto definitivo por síntese qualitativa. (SAÚDE, 2015)

1.5 Organização do trabalho

Este trabalho está dividido em 05 (cinco) capítulos definidos. O primeiro capítulo contém a parte introdutória, onde será definida a natureza e a solução proposta. Enquanto que no segundo capítulo contém a fundamentação teórica necessária para a justificativa do projeto. O capítulo três, por sua vez, terá intuito de informar a metodologia utilizada, detalhando todos os processos desenvolvidos durante o projeto. O quarto capítulo apresentará os resultados obtidos a partir da metodologia. E por fim, o quinto capítulo será informado as conclusões sobre o projeto.

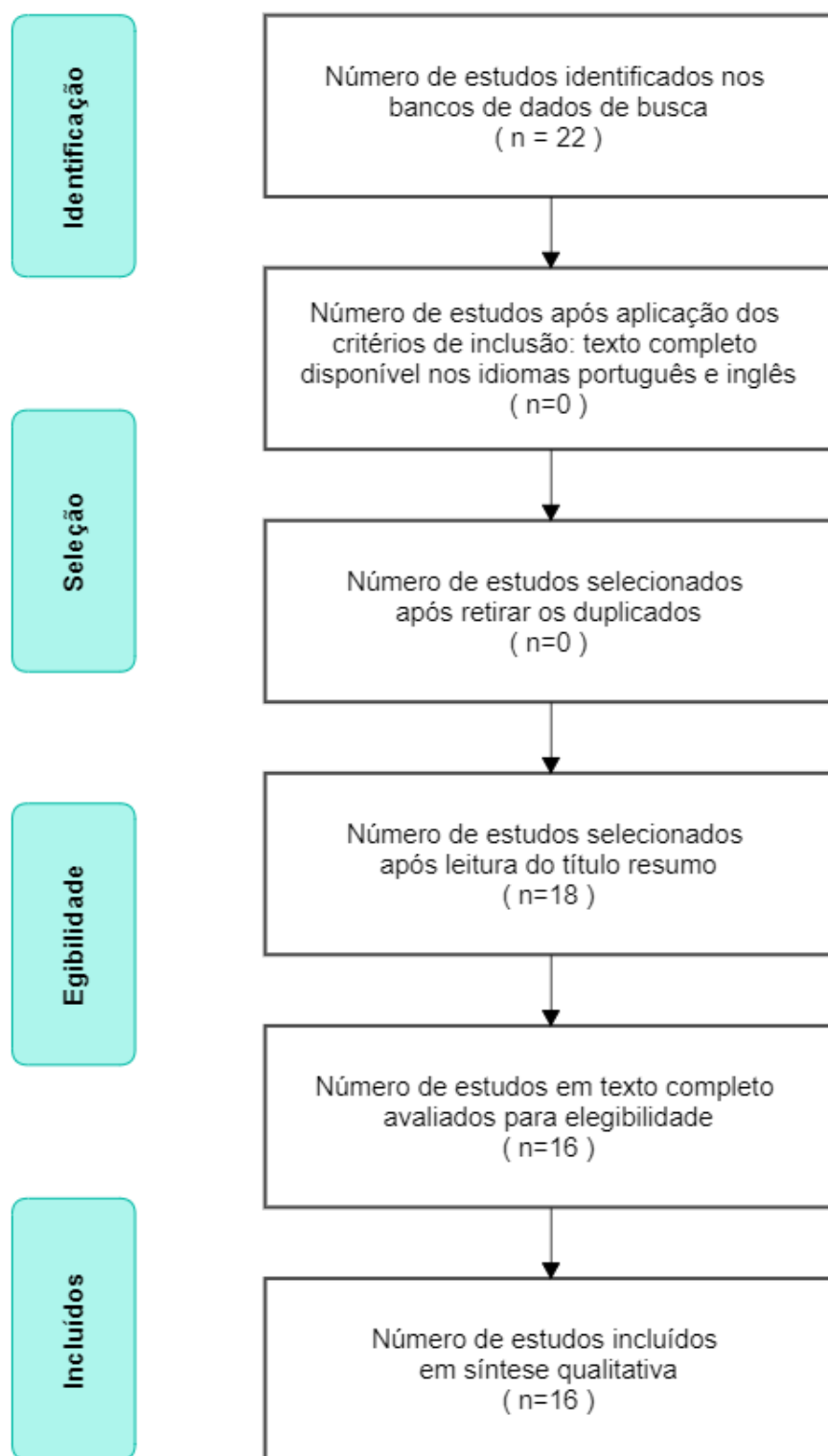


Figura 2 – Fluxograma PRISMA

2 Fundamentação teórica

Neste capítulo apresenta o embasamento teórico do projeto, tais quais as fórmulas e leitura complementar para que seja possível o planejamento e a execução de cada etapa definida em Objetivos na sessão 1.3

2.1 Processamento de imagens

A área de processamento de imagens é um dos assuntos de interesse por pesquisadores por permitir viabilizar grande número de aplicações em duas categorias bem distintas:

- Aperfeiçoamento de informações de imagens digitais para interpretação humana;
- Autômatos feitos por computador de informações extraídas de um ambiente ativo;

O processamento de imagens digitais é traduzido por meio de algoritmos. Em função disto, as principais funções de processamento de imagens podem ser implementadas via *software*, com algumas exceções nas etapas de aquisição e exibição. O uso de *hardware* especializado para processamento de imagens será necessário para automatizar e consolidar em um dispositivo de captação e processamento de imagens. (FILHO; NETO, 1999)

Para que uma imagem seja analisada e processada primeiramente é necessário um pré-processamento de imagem, condicionando e reconhecimento de um objeto estudado a partir da segmentação baseado pela forma do objeto fornecido em resultado a binarização e erosão, para continuar com o processamento de imagem que será analisado por meio da forma do objeto e armazenado em dados em um dispositivo.

No Diagrama 3 é detalhado o processo a ser implementado em cada etapa subsequente, onde é adquirido uma imagem real e é convertida em uma imagem em 2D (duas dimensões), onde ocorrerá um pré-processamento para ser segmentada de forma a analisar o histograma para fins de reconhecimento de formas em movimento.

2.1.1 Imagem digital bidimensional

Uma imagem digital pode ser descrita matematicamente por uma função $f(x, y)$ da intensidade luminosa, sendo seu valor, em qualquer ponto de coordenadas espaciais (x, y) , proporcional ao nível de tons de cinza daquela imagem naquele ponto. (FILHO; NETO, 1999)

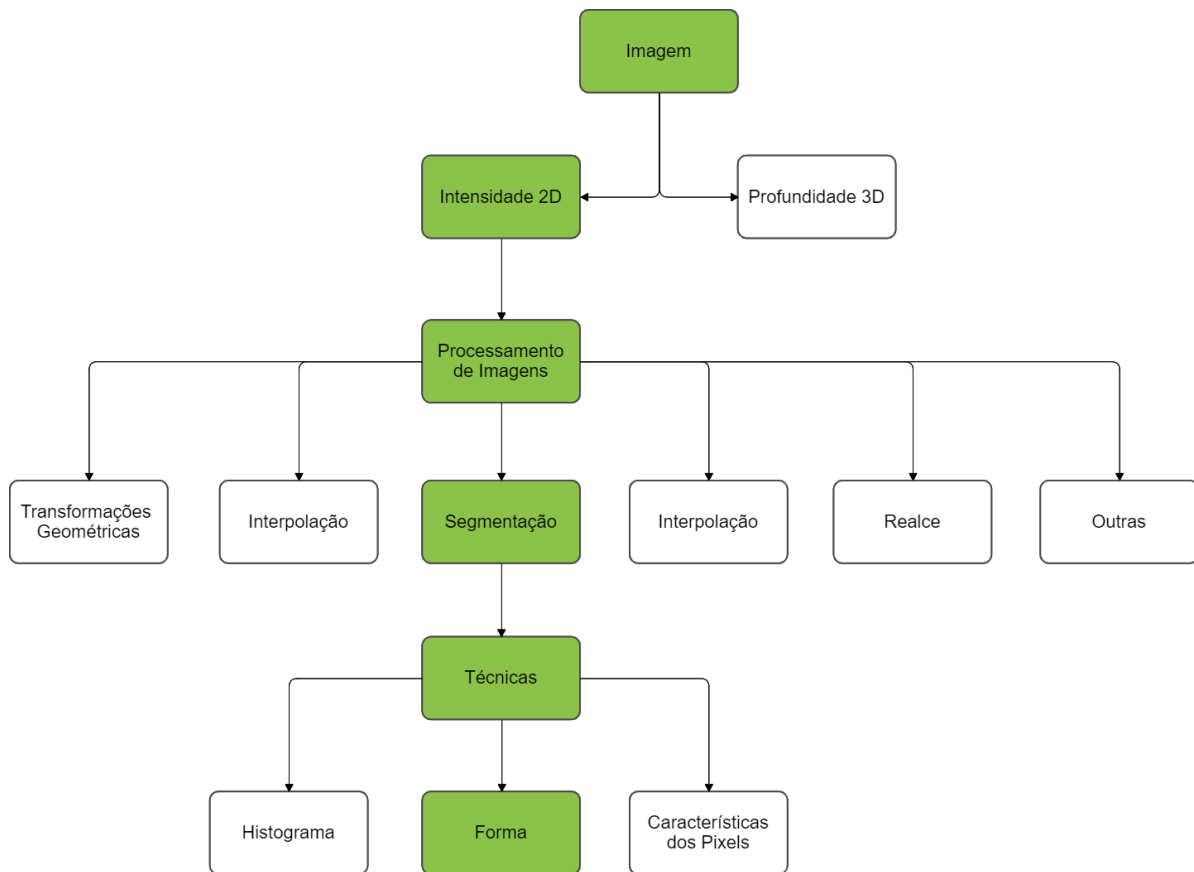


Figura 3 – Diagrama de processamento de imagens

No caso de uma imagem que possui informações em intervalos ou bandas distintas de frequência, é necessário uma função $f(x, y)$ para cada banda, no caso há diversos tipos de imagens, assim como citado na tabela 1:

Tipos de imagens	Características
Imagem binária	Matriz binária Valores entre 0 e 1 0 para preto, 1 para branco
Imagem monocromática	Matriz bidimensional Valores entre 0 e 255 Intensidade de <i>pixels</i> cinza
Imagem policromática	Matriz tridimensional 3 valores para cada <i>pixel</i> RGB Dimensão de L x C x 3

Tabela 1 – Tipos de Imagens (SOUSA, 2019)

Onde L e C é a resolução da imagem, linhas e colunas respectivamente.

No caso do projeto, a segmentação será trabalhada com imagens monocromáticas e binárias, para que uma imagem seja processada é fundamental representar sua informação adequado ao tratamento computacional.

2.1.2 Aquisição de imagem

Uma transdução optoeletrônica define-se em um processo de conversão de uma cena real tridimensional em uma imagem analógica bidimensional. A utilização da câmera *module v2* da Raspberry como uma câmera fotográfica converterá a cena 3D (três dimensões) em uma representação 2D adequada, similarmente da forma que [Scheidemantel \(2015\)](#) captou no projeto.

A Câmera Raspberry Pi é um dispositivo de captação de imagens modelo *OV5647 CMOS de 5 Megapixels* onde consiste de uma matriz de células semicondutoras fotossensíveis, que atuam como capacitores, armazenando carga potencial elétrica proporcional à energia luminosa incidente. O sinal elétrico produzido é condicionado por circuitos eletrônicos especializados, produzindo à saída um Sinal Composto de Vídeo (SCV) analógico e monocromático. ([LTD, 2021](#))

Um conjunto de prismas e filtros de cor encarregados de decompor a imagem colorida em suas componentes R (*Red*), G (*Green*) e B (*Blue*) é necessário para aquisição de imagens coloridas, cada qual capturada por um CCD (*Charge-coupled device*) independente. Os sinais elétricos correspondentes a cada componente são combinados posteriormente conforme o padrão de cor.

Uma câmera CCD monocromática simples consiste basicamente de um conjunto de lentes que focalizarão a imagem sobre a área fotossensível do CCD, o sensor CCD e seus circuitos complementares. ([FILHO; NETO, 1999](#))

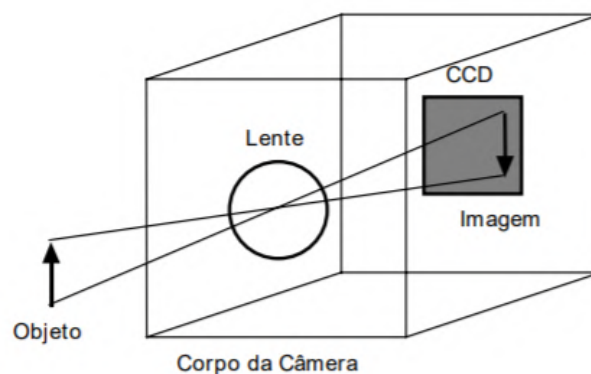


Figura 4 – Visão esquemática da câmera CCD

2.1.3 Digitalização

A digitalização consiste em uma conversão analógica-digital na qual o número de amostras do sinal contínuo por unidade de tempo indica a taxa de amostragem e o número de *bits* do conversor A/D (Analógico/Digital) utilizado determina o número de tons de cinza resultantes na imagem digitalizada. ([FILHO; NETO, 1999](#))

A imagem capturada é em RGB, composta pelas cores primárias vermelha, verde e azul, onde são combinados de várias formas de modo a reproduzir um largo espectro cromático. Contudo, o método de Otsu implementado no projeto necessita que a imagem seja monocromática, portanto, é necessário realizar o processo de conversão da imagem.



Figura 5 – Imagem policromática RGB

Uma imagem em RGB é composta por 3 elementos rearranjadas a partir de uma matriz $M \times N$, o qual cada valor da matriz é composta por 3 elementos de intensidade de 0 a 255. Com uma imagem monocromática, a matriz $f(x, y)$ apenas contém um valor de intensidade que compõe a imagem em 2D em escalas de tons de cinza.

$$f(x, y) = \begin{bmatrix} f(0, 0) & f(0, 1) & \dots & f(0, N - 1) \\ f(1, 0) & f(1, 1) & \dots & f(1, N - 1) \\ \vdots & \vdots & \ddots & \vdots \\ f(M - 1, 0) & f(M - 1, 1) & \dots & f(M - 1, N - 1) \end{bmatrix} \quad (2.1)$$

Altos valores de M e N indicam uma imagem de alta resolução por conter uma matriz com maiores elementos e mais detalhamento. (MONTEIRO, 2003)

2.1.4 Segmentação de imagens

O processo principal do projeto é o processamento de imagem por meio de segmentação em tempo real utilizando o método de segregação limiar de níveis de tons de cinza, onde varia de 0 a 255 em escala de luminosidade do *pixel* analisado, transformando uma imagem com esta variação de intensidade de *pixel* em uma imagem com apenas duas tonalidades, branca e preta, realçados de forma a separar a imagem com uma finalidade de interesse com base de um algoritmo que calcula o melhor valor limiar otimizado. (MONTEIRO, 2003)

O processo de segmentação de imagens decompõe uma imagem digital para que se possa analisar ela de forma a extrair apenas dados de interesse, como forma, cor e pela análise do histograma.

A Segmentação de imagens tem como objetivo obter um conjunto de regiões/objetos ou um conjunto de contornos extraídos de uma imagem. Cada um dos *pixels* em uma mesma região é similar com referência a alguma característica computacional como cor, intensidade, textura ou continuidade. Para que uma região não seja agregada a uma determinada característica, dados em fronteira devem respeitar características distintas.

2.1.5 Binarização

Por *Thresholding*, é possível segregar os *pixels* por uma representação binária de pela cor do *pixel* por preto ou branco por determinar uma faixa limiar constante da cor de intensidade de cinza do *pixel*, onde se for abaixo da intensidade constante é branco e se for acima é preto.

A função de Binarização $g(x, y)$ pode ser descrita em 2.2:

$$g(x, y) = \begin{cases} R_1, & \text{se } f(x, y) \leq T \\ R_2, & \text{se } f(x, y) > T \end{cases} \quad (2.2)$$

(formula)

Onde R_1 e R_2 são os valores estipulados para os níveis de cinza da imagem binarizada, no caso utiliza-se 0 (preto) e 255 (branco) e T para o valor de limiarização que segrega as duas imagens onde qualquer valor abaixo do valor T será considerado branco e o valor acima de T será considerado Preto.

O ponto de Limiarização T é a parte fundamental para segregar o objeto de estudo, para isso, é utilizado o método de Otsu para achar o valor de *Thresholding* ideal. (MONTEIRO, 2003)

A Figura 6 apresenta uma imagem em tons de cinza, sem nenhum processamento, que receberá um processo de binarização, o resultado pode ser visto na figura 7.

2.1.6 Histograma

O método a ser utilizado pelo projeto será pelo método do histograma com intuito de analisar os clusters de uma determinada imagem a partir de picos e vales por meio de cor ou intensidade.

O histograma é um gráfico onde é mostrado a intensidade do *pixel* a partir da frequência do *pixel* percorrido na imagem. A Figura 8 apresenta um exemplo de histograma calculado.



Figura 6 – Imagem em tons de cinza



Figura 7 – Imagem binarizada

A observação do histograma permite a localização do melhor valor de T para que seja separados os objetos do fundo. Esta localização é tão mais fácil quanto mais bimodal for o histograma. A partir do Histograma é possível analisar e ter uma estimativa de do ponto de corte de limiarização, onde há parte de vales entre as cristas dentro da faixa de meio-tons, ou seja, onde há a menor concentração dos *pixels* dentre as maiores frequência de intensidade de *pixels* para uma melhor segregação. (CLAYTON; NEVES, 2001)

Por meio do histograma será possível analisar se uma imagem é nitidamente aceitável para ser aprovada como uma imagem de qualidade caso apresentar níveis de tonalidades mais escuras indicando que há um elemento adicional na imagem para ser captado e recordado no programa.

A Figura 9 apresenta exemplos de histogramas onde é evidenciado o melhor valor

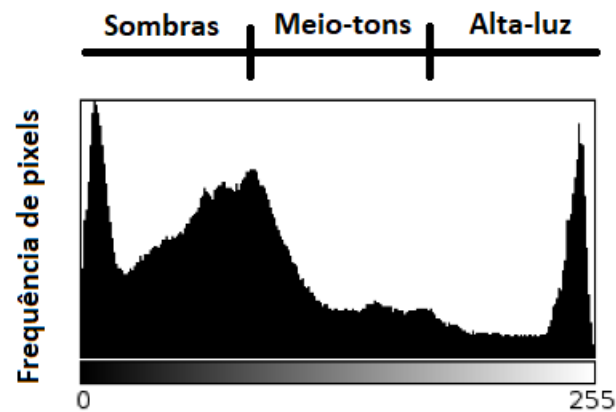


Figura 8 – Representação de um histograma

limiar dentre uma dispersão de *pixels* a partir de um objeto de interesse e o plano de fundo da imagem.

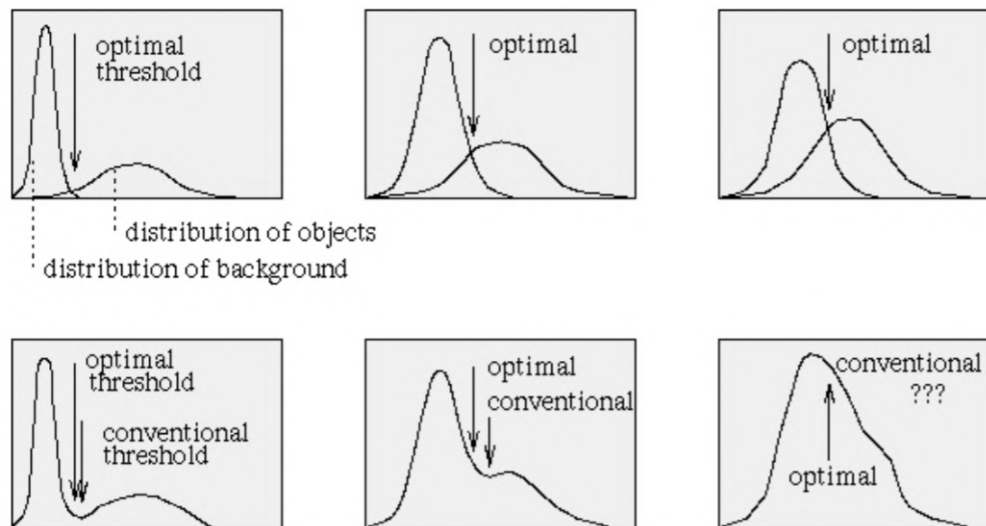


Figura 9 – Exemplos de otimização de histograma

2.1.7 Método de Otsu

O método de segmentação do Otsu é desejável nesse projeto pois com ele é possível ter uma otimização na detecção de movimento, visto que ele segmenta apenas objetos de interesse presente na imagens para fácil processamento, informação extraída a partir do artigo original de [Otsu \(1979\)](#).

O método de limiarização bimodal de Otsu se baseia na discriminação de uma imagem a partir da intensidade dos *pixels* em níveis de tons de cinza a partir de duas classes de tonalidade de cinza C_0 e C_1 onde são os valores do Objeto e do Fundo da

imagem em função de t onde t é o ponto de corte de limiarização da imagem. (CLAYTON; NEVES, 2001)

Seja,

$$C_0 = \{0, 1, 2, 3, \dots, t\}$$

$$C_1 = \{t + 1, t + 2, t + 3, \dots, l\}$$

σ_w^2 a variância dentro da classe

σ_b^2 a variância entre as classes

σ_t^2 a variância total.

A obtenção do melhor limiar é adquirido a partir de da maximização de uma das equações seguintes

$$\lambda = \frac{\sigma_b^2}{\sigma_w^2} \quad (2.3)$$

$$N = \frac{\sigma_b^2}{\sigma_t^2} \quad (2.4)$$

$$K = \frac{\sigma_t^2}{\sigma_w^2} \quad (2.5)$$

Para determinar as variáveis da função de limiarização, utiliza-se as seguintes funções de variância e desvio padrão

$$\sigma_t^2 = \sum_{i=0}^{l-1} (i - \mu t)^2 P_i \quad (2.6)$$

$$\mu t = \sum_{i=0}^{l-1} i P_i \quad (2.7)$$

$$\mu_t = \sum_{i=0}^t i P_i \quad (2.8)$$

$$\omega_0 = \sum_{i=0}^t P_i \quad (2.9)$$

$$\omega_1 = 1 - \omega_0 \quad (2.10)$$

$$\sigma_b^2 = \omega_0 \omega_1 (\mu_0 - \mu_1)^2 \quad (2.11)$$

$$\mu_0 = \frac{\mu_t}{\omega_0} \quad (2.12)$$

$$\mu_1 = \frac{\mu t - \mu_t}{1 - \omega_0} \quad (2.13)$$

$$P_i = \frac{n_i}{n} \quad (2.14)$$

Onde P_i é a probabilidade de ocorrência do nível de cinza i , n_i é o número de *pixels* como o nível de cinza i e n é o número total de *pixels* de uma dada imagem definida como:

$$\sum_{i=0}^{l-1} n_i \quad (2.15)$$

A partir do ponto de corte de limiarização t de uma imagem as probabilidades de ω_0 e ω_1 indicam a região das áreas ocupadas pelas classes C_0 e C_1 . As médias μ_0 e μ_1 são a média de níveis de cinza da imagem original. O valor máximo de n pode ser utilizado como um indicativo de separabilidade a partir das classes C_0 e C_1 da imagem ou na bimodalidade do Histograma. O fator n varia de 0 a 1, onde 0 indica que a imagem tem apenas uma tonalidade de cinza e 1 informa que a imagem tem apenas duas tonalidades de cinza, o que indica um tipo de segmentação 100% eficiente. (MONTEIRO, 2003)

2.1.8 Filtro *GaussianBlur*

Filtros são técnicas de processamento que tem por finalidade realçar determinadas características em imagens digitais ou reduzir ruídos. Esses ruídos presente nas imagens surgem a partir da forma de aquisição da imagem, devido a limitações de hardware, seja no processo de quantização e digitalização, pela forma de compressão da imagem e pelos problemas na transmissão

O filtro Gaussiano é utilizado como um filtro passa-baixa, discretizado a partir de uma função Gaussiana de duas dimensões para obtenção de valores mascarados definida digitalmente

$$G = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (2.16)$$

onde o σ é o desvio padrão. Essa distribuição tem a característica conforme a figura 10, assumido que a distribuição tem média zero (i.e. está centrada em $x=0$ e $y=0$). A distribuição para $\sigma = 1$.

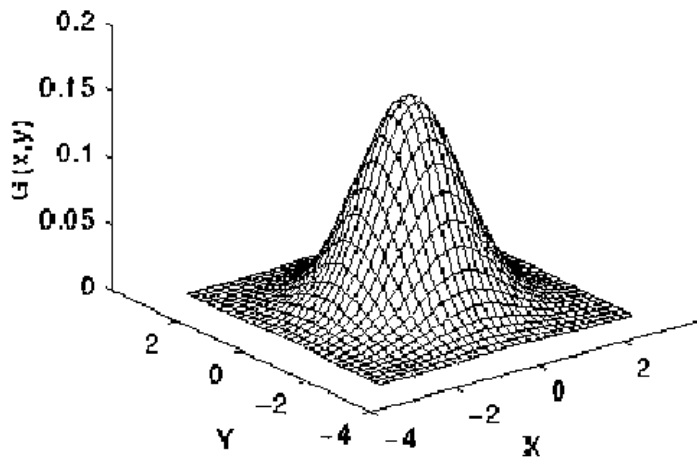


Figura 10 – Forma da distribuição Gaussiana em 2D média (0,0) e desvio padrão 1

A imagem é armazenada como uma coleção de pixels discretos, é necessário produzir a forma discreta da distribuição Gaussiana para a obtenção de um núcleo de convolução. O efeito do filtro Gaussiano é suavizar (*smoothing*, *blur*) a imagem, de forma que o resultado seja suave proporcionalmente ao desvio padrão da Gaussiana utilizada. O impacto do desvio padrão maior também faz com que a máscara acompanha com a representação adequada.

2.2 Detecção de movimento

Em física, o movimento consiste no deslocamento de posição de um corpo ou de um sistema em relação ao tempo, medido por um dado observador num referencial determinado em um determinado ponto de partida.

2.2.1 Subtração de frames

O princípio da identificação de movimentos utilizado é a subtração de frames, onde consiste em capturar um frame e compará-lo com o frame anterior subtraindo os pixels de uma imagem da outra anteriormente capturada. Dessa forma, todas as partes das imagens imóveis serão anuladas. Como as imagens que estarão iguais são as que não mudaram de lugar, de forma a evidenciar o local onde ocorreu o movimento, observado na Figura 11. O método concomitantemente utilizado por [Bruxel \(2015\)](#), o qual fez o mesmo procedimento, segmentou e comparou a diferença de frame do atual com a anterior.



Figura 11 – Imagem fornecida por diferenças de frames captadas

A biblioteca OpenCV oferece uma função que subtrai duas imagens, é a **absdiff()**. A função recebe duas imagens em forma de array e retorna sua diferença absoluta dentre as imagens comparadas, biblioteca onde [Pereira e Oliveira \(2016\)](#) utilizou para compor o projeto.

Algumas variáveis externas podem fornecer ruídos para o sistema, portanto é necessário analisar fornecer as seguintes recomendações: ([SINGLA, 2014](#))

- Posicionar a câmera em um local estável e inerte de movimento
- Iluminação estável, iluminação adequada

- Alto contraste entre o objeto e o fundo da imagem
- Câmera com alta resolução e alta taxa captação de frames

2.3 Métrica de Avaliação

Uma função vital do sistema de detecção de movimento proposto é a integridade do sistema, no caso se o projeto proposto apresenta funcionamento e resultados constantes, de tal forma que seja previsto a vida útil do serviço. (AHMED; ECHI, 2021)

A qualidade do trabalho está na eficiência de detectar objetos em movimento e armazená-los no dispositivo, portanto, uma forma de qualificar o sistema é quantificar as vezes o qual sistema detecta ou não objetos em movimento, ou detectar movimento na imagem em sistema estático. Para isso, será utilizado o método qualitativo de previsão de classes, no caso será utilizado o conceito de verdadeiro/falso e positivo/negativo. (DINIZ, 2017)

- Verdadeiro positivo (VP): Detecção de movimento correta do programa sob objeto dinâmico.
- Verdadeiro negativo (VN): Não ocorre a detecção de movimento quando não há movimento observado.
- Falso positivo (FP): Detecção de movimento incorreta do programa sob objeto estático.
- Falso negativo (FN): Não ocorre a detecção de movimento quando não há movimento observado.

2.3.1 *F-measure*

F-measure é um método de medida o qual avalia o desempenho do comportamento de amostras, de forma a ser calculado pela equação 2.17, onde esse parâmetro varia entre zero a um, de forma quanto mais próximo de um, melhor e mais eficiente é o método utilizado.

$$F = 2 \cdot \frac{precision \cdot recall}{precision + recall} \quad (2.17)$$

A variável *precision* informa a taxa de amostras coletadas relevantes e é calculada a partir de números de verdadeiro positivos e falsos positivos conforme a equação

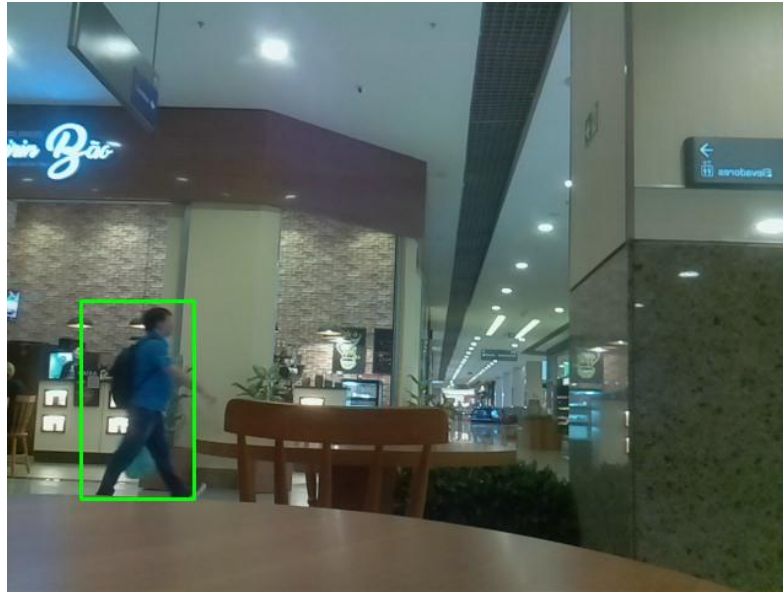


Figura 12 – Verdadeiro positivo



Figura 13 – Verdadeiro negativo

$$precision = \frac{VP}{VP + FP} \quad (2.18)$$

A variável *recall* informa a taxa de amostras relevantes as quais foram coletadas e é calculada a partir de números de verdadeiro positivos e falso negativos conforme a equação

$$recall = \frac{VP}{VP + FN} \quad (2.19)$$

(GRIGATI; DIAS, 2015)



Figura 14 – Falso positivo



Figura 15 – Falso negativo

2.4 Hardware

2.4.1 Raspberry Pi 3 Model B

A Raspberry é um dispositivo microcontrolador ou um mini-PC utilizado em projetos embarcados a baixo custo, com preço lançado a U\$35,00, encontra-se mais em conta aqui no Brasil na faixa de R\$200,00, por isso, é um dos periféricos mais completos e visados por desenvolvedores.

Com processador integrado Broadcom BCM2837 *quad-core* de 1,2 GHz, a Raspberry consegue instalar sistemas operacionais como Windows e Linux, de forma a operar

identicamente a um computador. Com um processador gráfico *dual core OpenGL* de 24 GFLOPs permite a utilização dela em projetos mais complexos como de processamento de imagens, pois como o trabalho de Becker (2018) utiliza um robô de competição com identificação visual de alvos.

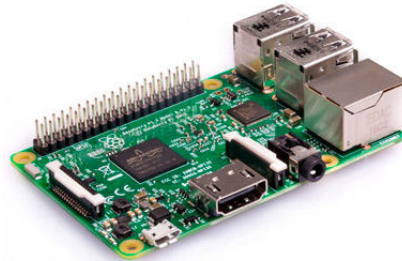


Figura 16 – Raspberry Pi 3 Model B

2.4.2 Especificações da Raspberry Pi 3 Model B

As especificações da Raspberry Pi3 Model B microcontroladora é definida a partir da tabela 2: (GUIDE, 2021)

Especificações	Raspberry Pi 3 Model B
Processador Quad-core (GHz)	1.2
Memória RAM (GB)	1
Dimensões (mm)	85 x 56 x 17
Alimentação (v)	5
Nº de pinos	40
Frequência do <i>Clock</i> (MHz)	400
Wi-Fi (b/g/n)	802.11

Tabela 2 – Tabela de especificações da Raspberry Pi 3 Model B

2.4.3 Camera Module Omnivision OV5647

A escolha da câmera modular V2 foi decidido pelo produto mais barato no mercado compatível com o microcontrolador no valor de R\$50,00, pois o objetivo dessa monografia é ser o mais rentável e otimizado possível.

A câmera possui um sensor OmniVision OV5647 Cor CMOS QSXGA 5 Megapixels com gravação de vídeo em *full HD* 1080p a uma taxa de 30 FPS. A forma o qual os dois dispositivos se conectam por um cabo flat pelo protocolo de comunicação I2C (*Inter-Integrated Circuit*) o qual informa os comandos de captação de imagem e por meio da



Figura 17 – Câmera Module V2 OmniVision OV5647

conexão CSI (*Camera Serial Interface*) envia dados para um receptor de sinal *Unicam*, onde converte o sinal em forma de pixels para ser enviado à central de processamento *libcamera*. (LTD, 2021)

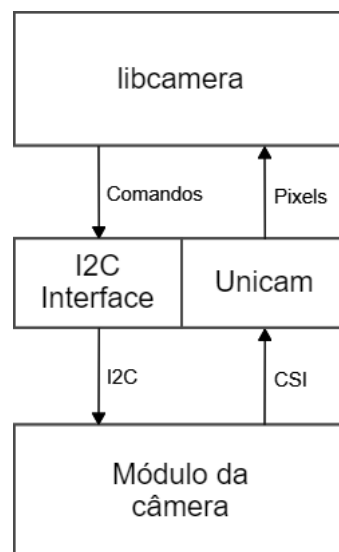


Figura 18 – Esquemático comunicação da câmera

2.4.4 Lâmpada LED Elgin

Para o experimento de teste de captação de imagem em ambientes com luminosidade variável foi utilizado a lâmpada inteligente com controle de cores e intensidade luminosa via *bluetooth* e *Wi-Fi*. A Figura 19 representa a lâmpada utilizada e a Tabela 3 apresenta as especificações do objeto. (INTELBRAS, 2022)

Especificações	Lâmpada Elgin
Potência	7 watts
Voltagem	100-240 Volts
Fluxo luminoso	806 lm
Cores	RGB
Fabricante	Intelbras

Tabela 3 – Especificações da lâmpada Elgin



Figura 19 – Lâmpada inteligente Elgin

2.5 Software

A programação completo do projeto será feita na Raspberry, visto que possui poder de processamento de um minicomputador, a linguagem utilizada para codificação dos algoritmos foi a linguagem de programação em Python, responsável pela configuração dos protocolos, controle de periféricos e método de segmentação de Otsu e a construção da biblioteca de código OpenCV. (BECKER, 2018)

2.5.1 OpenCV

A biblioteca de código pública OpenCV é responsável por abranger projetos de visão computacional e aprendizagem robótica com funcionamento em sistemas operacionais utilizados pela Raspberry, como a do Linux, Windows, MAC OS e interface em Java, promovido para assistir desenvolvedores de projeto pela vasta aplicação computacional e em tempo real. (OPENCV, 2019)

A vasta biblioteca de aproximadamente 2500 algoritmos permite aplicações em diversas finalidades. Muito utilizado em empresas e desenvolvedores, essa biblioteca permite trabalhar em diversos módulos em formato de pacotes de códigos, dentre eles é informado na tabela 4. (SOUSA, 2019)

Módulos	Função
<i>Core</i>	Estrutura fundamental de dados e funções básicas dos módulos
<i>Imgproc</i>	Técnicas de processamento de imagens
<i>Video</i>	Análise de vídeo
<i>Videoio</i>	Interface para captura de codecs de vídeo
<i>Objdetect</i>	Detecção de objetos e características de classes desejadas
<i>Highgui</i>	Interface de desenvolvedor de recursos de UI
<i>Calib3d</i>	Calibração da câmera e reconstrução 3D
<i>Features2d</i>	Detecção e descritores de pontos de saliência

Tabela 4 – Módulos e funções da biblioteca OpenCV

2.6 Captação em *streaming*

A captura da imagem por meio da câmera integrada da Raspberry é feita de forma remota por *streaming*. O codificador *stream* elementar é responsável pela transmissão do vídeo captado e comprimido em MPEG para o servidor, de forma a se organizar em blocos de dados nomeados de *packets*, onde combinados de forma quântica gera um PES. Um fator determinante da qualidade da transmissão é a latência do vídeo, onde é o tempo o qual a imagem é captada e transmitida até o espectador, portanto, quanto melhor o processamento do dispositivo a enviar a imagem, mais baixa é a latência.

O PES é multiplexado por um multiplexador antes de ser transmitido, transformando-o em um *transport stream*, o qual contém os pacotes relativos ao vídeo comprimido e os metadados relativos da própria stream. Múltiplos vídeos conseguem ser multiplexados em único *transport stream*, pois ocorre uma transmissão de uma tabela de associação de programas, de forma a relacionar todos os vídeos transmitidos na stream. Os pacotes contidos no *transport stream* possui tamanho de 188 bytes e carrega um código de identificação de programa. Dessa forma, os pacotes de *transport stream* traduzidos pelo decodificador garante uma eficácia de sincronização. O fluxo de streaming de vídeo é demonstrado na figura 20 a partir dos conceitos básicos de streaming de vídeo. (SCHEIDEMANTEL, 2015)

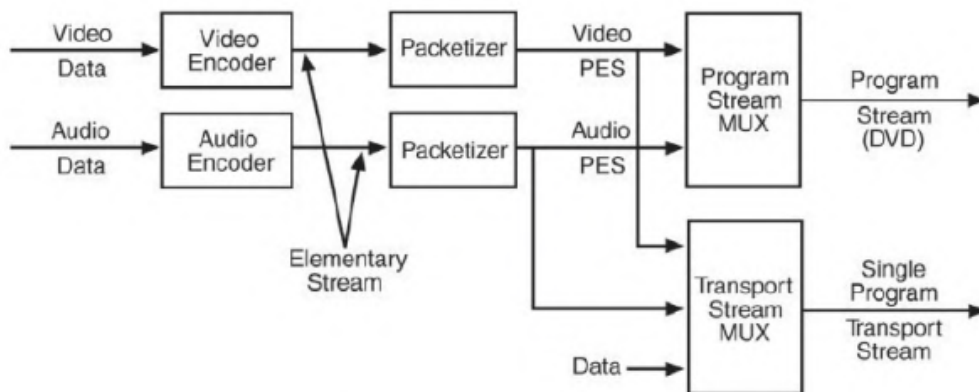


Figura 20 – Fluxo da codificação MPEG

3 Metodologia

Neste capítulo será apresentado a metodologia aplicada para a realização desse projeto, de forma a atingir os objetivos gerais e específicos.

Para o projeto, a Raspberry foi posicionada em locais simulando o funcionamento do sistema de detecção de movimento, de forma a pegar aplicação da captação de imagens local e via *streaming*.



Figura 21 – Raspberry Pi 3 e Câmera

Na execução do código de captação de imagens diretamente na raspberry, inicia-se abrindo três janelas para a verificação do comportamento do algoritmo e com a finalidade de processá-las para a corroboração dos objetivos propostos:

- Captação da imagem policromática em tempo real (figura 22);
- Captação da imagem segmentada binarizada em tempo real (figura 23);
- Captação da variação das posições dos frames em valor absoluto (figura 11);

3.1 Algoritmo

3.1.1 Captação de imagem em tempo real

A captura da imagem por meio da câmera integrada da Raspberry é feita a partir do componente integrado câmera da Raspberry. Para isso, foi feito testes utilizando a



Figura 22 – Imagem captada colorida



Figura 23 – Imagem captada segmentada

funcionalidade da câmera para analisar a qualidade de imagem fotografada a 5 megapixels com o comando **raspistill** no formato JPEG.

A forma de capturar a imagem de forma automatizada e otimizada a partir de um código em Python com a utilização da biblioteca **cv2** com o comando **cv2.VideoCapture** para captação da imagem fornecida pela câmera. Exemplo mostrado na figura 22.

Para a captação da imagem em *streaming*, é utilizado um código de geração de um servidor via *flask*, método utilizado pelo Kouao (2020), o qual coleta o IP *local host* e transmite para um navegador de internet, dessa forma pode ser utilizado por pessoas remotamente, conforme a figura 24 apresentada.

3.1.2 Cálculo do *Threshold* de Otsu

Após a captação da imagem colorida, é necessário convertê-la em tons de cinza para que seja possível segmentá-la, portanto, a imagem policromática é convertida em um array

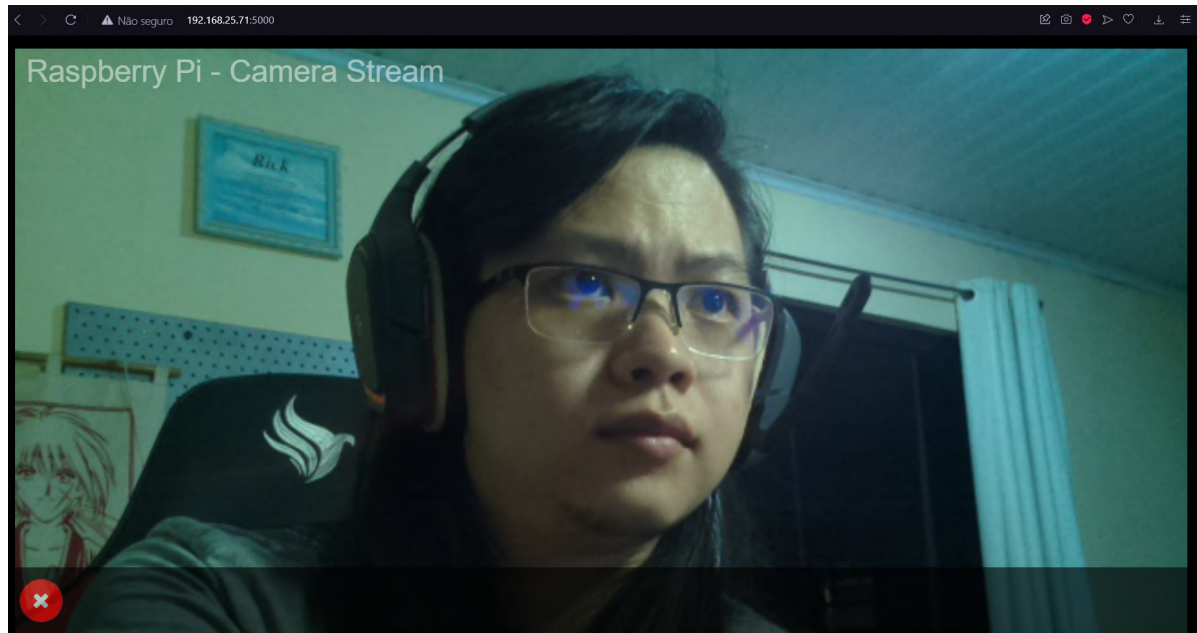


Figura 24 – Interface do navegador com a captação de imagem via *streaming*

composto por números de 0 a 256 em níveis de cinza com a função `cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)`, resultado apresentado de acordo com a figura 25.



Figura 25 – Imagem captada em escala de cinza

Aplica-se o filtro Gaussiano da sessão 2.1.8 para remoção de pequenos ruídos e interferência com a função `cv2.GaussianBlur(gray, (5, 5), 0)`.

Baseado nas equações apresentadas na 2.1.7, é declarado uma função onde se calcula o limiar ótimo utilizando o método de Otsu, onde é atualizado automaticamente de acordo com a imagem captada e aplicando o limiar (`thresh_calculated`) na função `thresh = cv2.threshold (frameDelta, thresh_calculated, 255, cv2.THRESH_BINARY)`. Observa-se a aplicação do limiar como resultado apresentado na figura 23.

3.1.3 Cálculo do histograma

O histograma foi adquirido a partir da imagem captada quando há movimento, pois quando se inicia a gravação da imagem, é captado um frame da entidade em movimento, de forma a extrair a imagem convertida em tons de cinza para o cálculo do hitograma utilizando a função `cv2.calcHist([gray], [0], None, [256], [0, 256])`, exemplificada na figura 26.

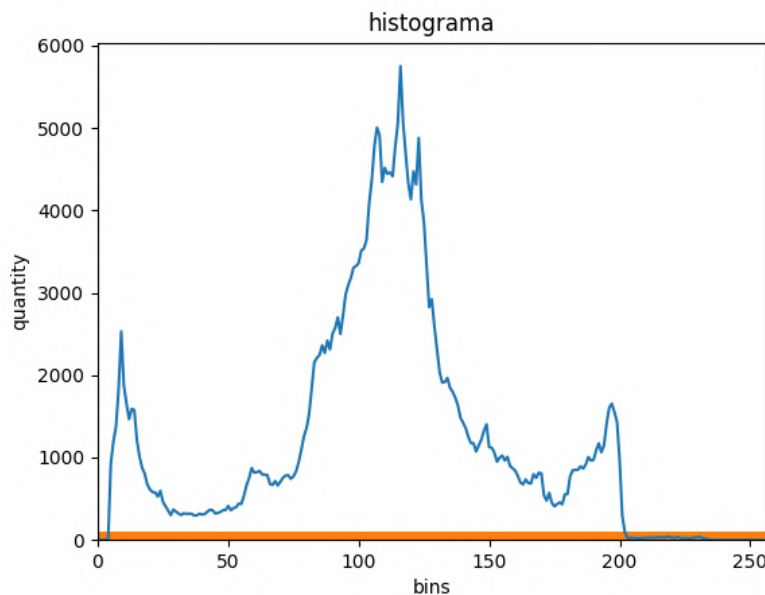


Figura 26 – Histograma gerado pelo algoritmo

3.2 Gravação de objetos em movimento

Conforme a dita sessão 2.2, o responsável pela detecção de movimento é o array **frameDelta**, onde é calculado pela diferença absoluta da imagem em níveis de tons de cinza inicial (**first_frame**) com a final (**gray**) fornecida pela função `cv2.absdiff(first_frame, gray)` representada pela figura 11. A função responsável pela gravação de vídeo no formato **avi** é a **writer** = `cv2.VideoWriter(filename,fourcc,FPS,(width, height),has_color)`, a qual agrupa os frames de interesse e os comprime em um arquivo de vídeo, onde os parâmetros são:

- Filename: Informa o nome do arquivo de vídeo gerado;
- fourcc: Fornece o formato de codificação MPEG;
- FPS: A taxa de gravação frames por segundo;
- Width, height: A altura e largura do vídeo;

O movimento captado está na representação de pequenos retângulos verdes onde está ocorrendo a variação de frames, baseando-se nas imagens segmentadas para fácil visualização de cada frame observado com a função `cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`. É possível observar os contornos na figura 28 em volta da pessoa em movimento.

3.2.1 Diagrama de máquina de estado

Com auxílio do diagrama de máquina de estado da figura 27, inicialmente é verificado a média dos valores do array **frameDelta**, este valor é inserido como o valor de referência informando o estado de gravação "sem movimento", onde permanece quando a imagem se apresenta estática. Caso haja uma variação da média de valores do array do **frameDelta** significativamente maior que a de estado "sem movimento", o programa capta o primeiro frame a partir da detecção de movimento, uma imagem é policromática com contornos verdes indicando onde está ocorrendo o movimento (figura 28) e a outra é uma imagem segmentada (figura 29) a partir da geração do histograma da imagem (figura 26). Após a captação dessas imagens, o código começa a fazer uma gravação do objeto em movimento até a condição de parada, no caso após 50 frames finais após a imagem não captar nenhuma variação de movimento caso o objeto fique estático ou fora da visão da câmera. Após o fim da gravação, ocorre então a condição de checagem de detecção de movimento com o antigo valor de média da variável **frameDelta** e retorna para a condição de estado "sem movimento".

A condição de iniciar a gravação de 50 frames é alterável de acordo com a preferência do usuário ou da frequencial do *clock* da Raspberry no momento o qual está sendo verificada no funcionamento do programa.

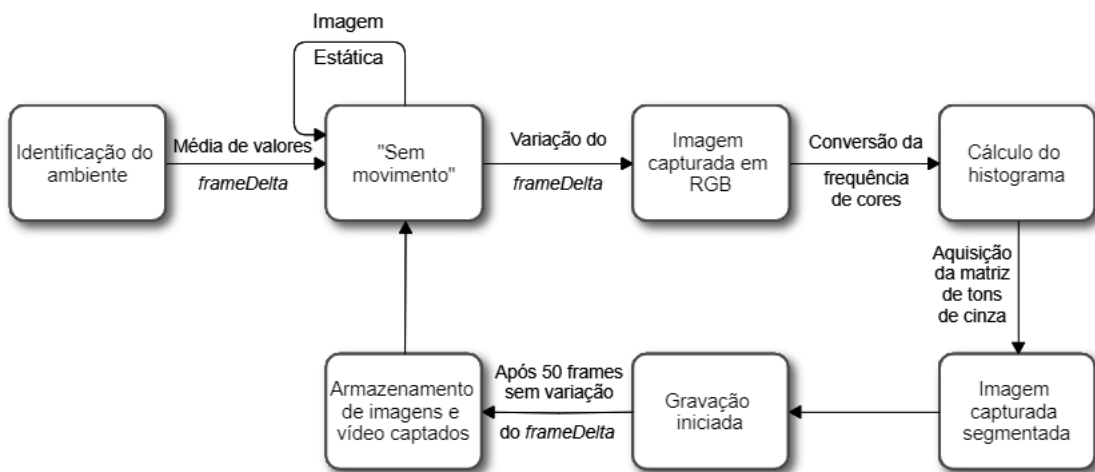


Figura 27 – Diagrama de máquina de estado da detecção de movimento

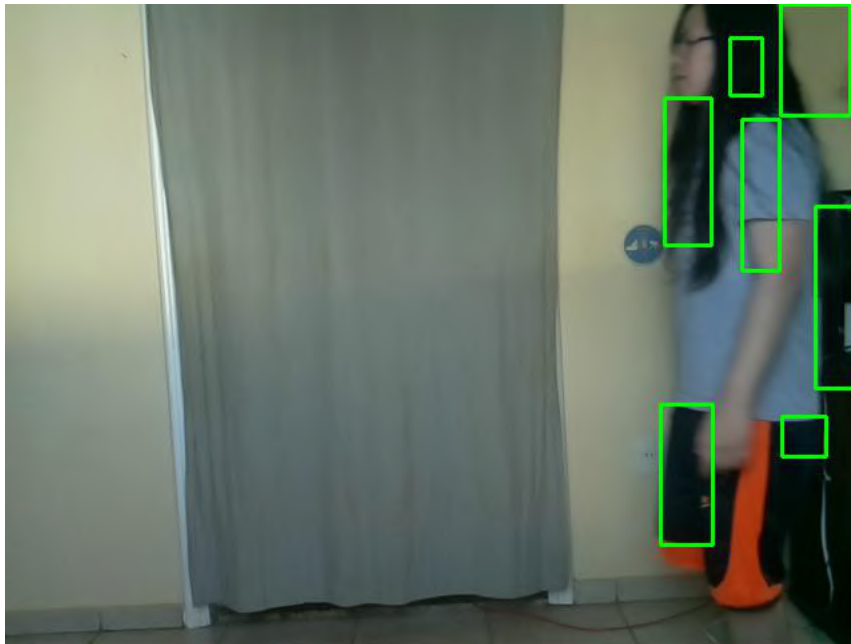


Figura 28 – Imagem captada objeto em movimento



Figura 29 – Imagem captada objeto em movimento segmentada

4 Resultados

Neste capítulo será tratado na análise da metodologia aplicada para ser discutido se os objetivos gerais e específicos foram atendidos para conclusão do projeto.

4.1 Captação da imagem em movimento

O responsável pela captação da imagem foi a câmera da Raspberry com uma resolução de 640 x 480 pixels. As imagens trabalhadas na Metodologia apresentou ter ótimo desempenho, visto que a qualidade da imagem permitiu identificar os objetos em movimento.

Sobre o tamanho de cada imagem armazenada no dispositivo de armazenamento, a imagem policromática representada na figura 22 possui o tamanho de 168 *Kbytes*, em comparação a imagem captada segmentada na figura 23, o qual tem o tamanho de 12,5 *Kbytes*.

Um fator analisado para a qualidade de captura da imagem foi a latência da captação dos vídeos das duas formas distintas: A taxa média de quadros por segundo calculada diretamente da Raspberry foi de 10 FPS, taxa esperada pela própria fabricante de acordo com a sessão 2.4.3. Contudo, essa taxa houve queda para 1 FPS quando vai *streaming*, uma queda de 90%, mas tal queda não interferiu na qualidade da imagem captada mesmo com uma latência tão alta, pois há um coparativo com a figura 28, captada diretamente da Raspberry, e a figura 30, captada via *streaming*. A qualidade da gravação também não foi prejudicada como pode ser vista no vídeo disponível na plataforma *Youtube* em Yamamoto (2022b).

O código implementado segue a lógica de acordo com o diagrama de máquina de estado referente a figura 27, de forma a tal gravar apenas quando se há movimento diante da imagem captada.

Com intuito de confirmar o funcionamento do algoritmo, o vídeo de gravação de detecção de movimento diretamente na Raspberry se encontra na referência Yamamoto (2022a), concomitantemente o vídeo capturado via *streaming* em Yamamoto (2022b).

Como descrito no Capítulo 2.3.1, para a validação do enunciado proposto, foram feitos testes de eficiência do sistema, de tal forma a analisar e mensurar a quantidade de imagens as quais compactua do conceito de *F-measure*.

Os objetos de interesse a serem analisados foram divididos em pessoas, objetos e sombras; para cada uma variável, foi atribuído os valores de *F-measure*, *recall* e *precision*,



Figura 30 – Imagem captada objeto em movimento via *streaming*

conforme a tabela 5

Locais	VP	FP	FN	<i>precision</i>	<i>recall</i>	<i>F-measure</i>
DF Plaza	39	3	2	92,8%	95,1%	96,2%
Taguatinga Shopping	95	10	16	90,4%	85,5%	87,9%
Rua período noturno	77	14	22	84,6%	77,8%	81,1%

Tabela 5 – Resultados coletados de locais observados

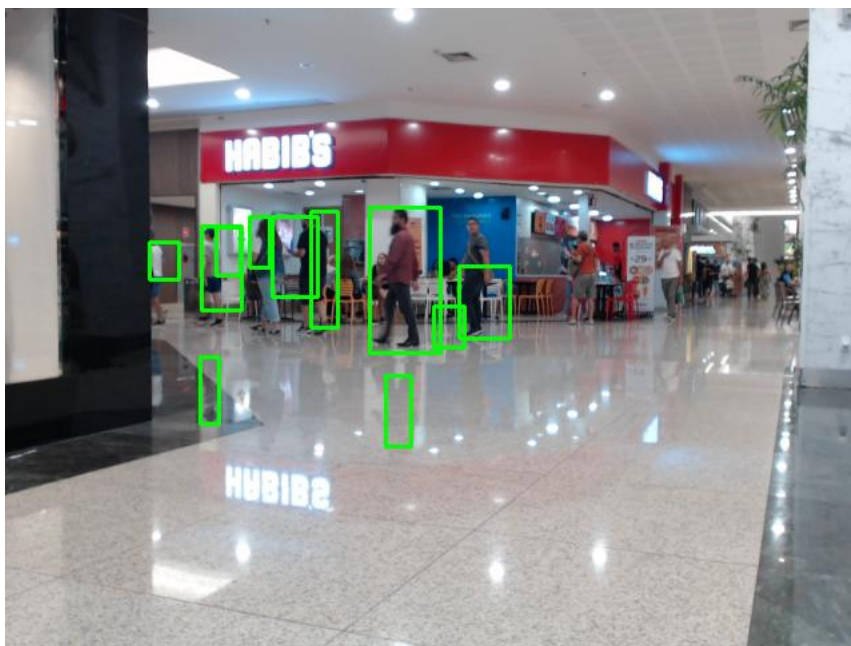


Figura 31 – Imagem coletada de Taguatinga Shopping

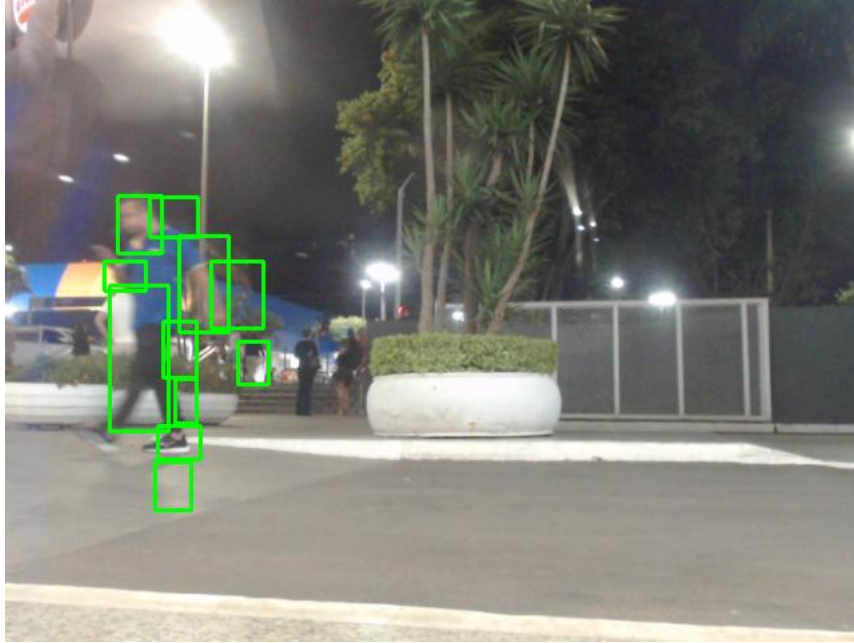


Figura 32 – Imagem coletada de rua período noturno

4.1.1 Captação de pequenos objetos

A captação de objetos menores em movimento foi detectável em um ambiente com vários detalhes, de forma a analisar o comportamento do algoritmo diante de ruídos, de tal forma utilizando uma pequena equação de comparação de tamanhos de tela analisada,

$$Captação = \frac{[Altura] \times [Largura](ImagemMovimentada)}{[Altura] \times [Largura](ImagemTotal)} \quad (4.1)$$

Pelos cálculos, abaixo de 4% da imagem captada em movimento em relação a imagem total não é detectável pelo programa, visto que a variação de pixels é inferior ao parâmetro de diferença de frames utilizado, onde a figura 33 indica o valor mínimo de movimento perceptível ao programa.

4.1.2 Captação em luminosidade variável

Para efeitos comparativos, foi feito um experimento de coleta de imagem em intensidade de luminosidade variável. Para esse experimento, foi utilizado um objeto o qual era preso sob o teto e com um movimento pendular foi captado diversos dados para análise qualitativa do projeto.

Para uma análise mais completa, foi utilizado a lâmpada Elgin detalhado no Capítulo 2.4.4 para fazer o controle de luminosidade por meio de intensidade onde por meio do aplicativo é possível regular a intensidade da luminosidade o qual varia de 1% a 100%, onde 100% é equivalente a 7 watts de potência.

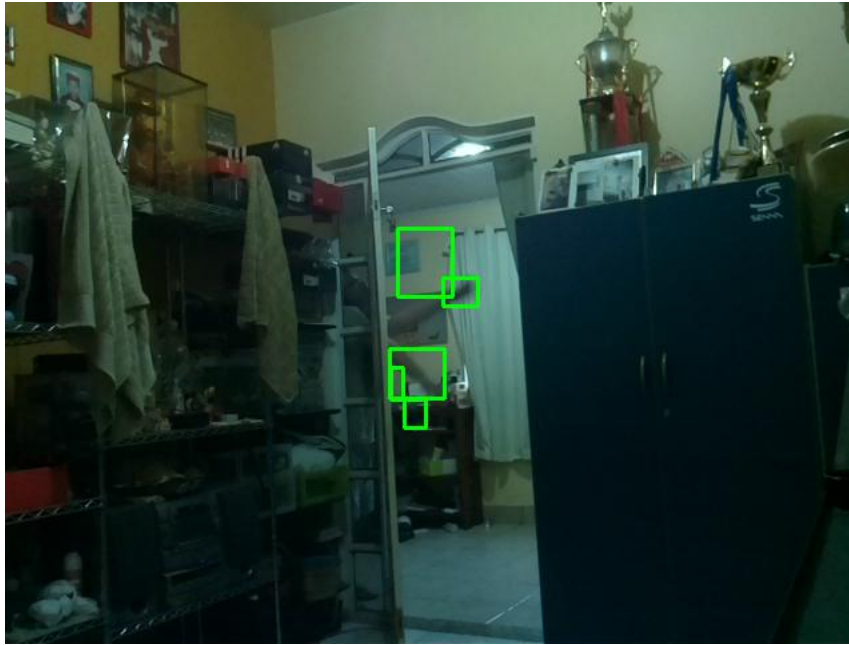


Figura 33 – Imagem captada em 4% em movimento em relação à imagem total

O experimento realizado foi realizado acoplando um objeto sob influência de um cordão o qual foi atado em uma superfície superior para simular um movimento pendular, para que o comportamento do objeto seja sempre o mesmo na realização dos experimentos, semelhante ao esquemático da Figura 34

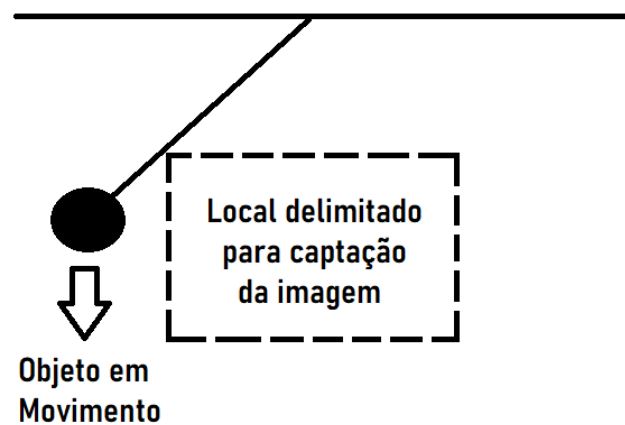


Figura 34 – Esquemático do experimento movimento pendular

O experimento foi feito dividido em 4 níveis de intensidade luminosa, de forma que a imagem captada em nível de intensidade a 1% na figura 35, a 33% na figura 36, a 66% na figura 37 e a 100% na figura 38.

A tabela 6 informa os dados de *F-measure* direcionados aos níveis de intensidade variadas a partir da coleta de dados pela Raspberry.

Níveis de luminosidade	VP	FP	FN	<i>precision</i>	<i>recall</i>	<i>F-measure</i>
1%	41	4	15	91,1%	73,2%	81,2%
33%	52	2	6	96,3%	89,6%	92,8%
66%	55	1	3	98,2%	94,8%	96,5%
100%	54	1	2	98,2%	96,4%	97,6%

Tabela 6 – Resultados coletados de luminosidade variável



Figura 35 – Imagem captada em 1% de intensidade luminosa



Figura 36 – Imagem captada em 33% de intensidade luminosa

4.2 *Threshold de Otsu*

O algoritmo de fato implementa a segmentação desejada, separando objetos de interesse com as características similares, como pode ser observado nas figuras 23 e 29.

Para confirmar se de fato a segmentação de Otsu está correta, foi utilizado uma imagem capturada da câmera com discrepância de quantidade a níveis de tons de cinza,

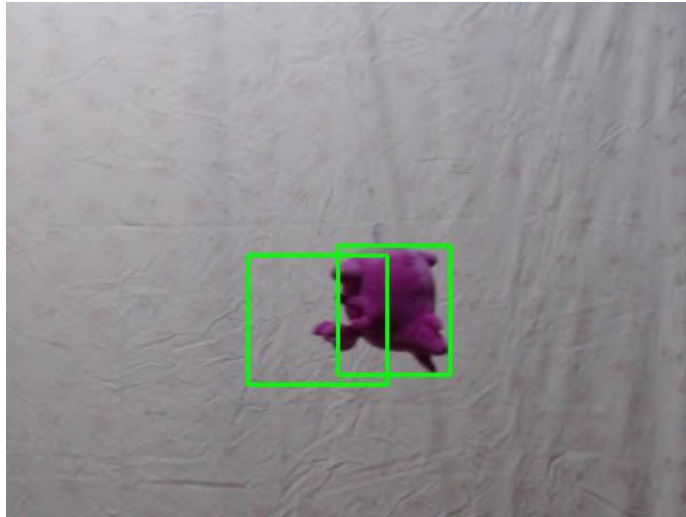


Figura 37 – Imagem captada em 66% de intensidade luminosa

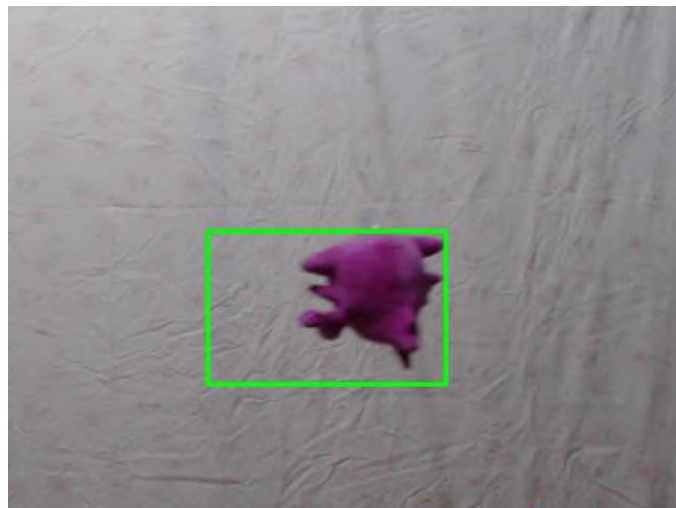


Figura 38 – Imagem captada em 100% de intensidade luminosa

caracterizado pelo histograma da figura 42. Como é observável, foi utilizado um objeto branco em um fundo preto (figura 39) segmentada e consideravelmente evidenciada e segregada do fundo, de acordo com a figura 40 apresentada.

4.3 Histograma gerado

Com um complemento de análise, o programa também gera um histograma no formato de arquivo de imagem JPEG extraída a partir do frame capturado em movimento em escalas de tons de cinza.

Como parâmetro, foi utilizado o mesmo experimento similar ao esquemático da Figura 34, contudo, o diferencial neste experimento foi a utilização de um objeto com uma cor com alto contraste do fundo. A imagem coletada está sendo representada na

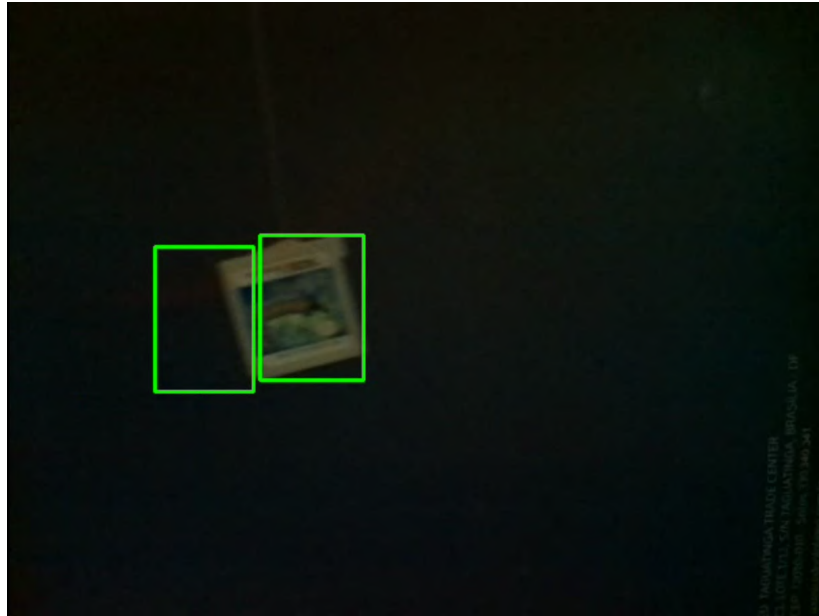


Figura 39 – Exemplo de imagem policromática captada com fundo

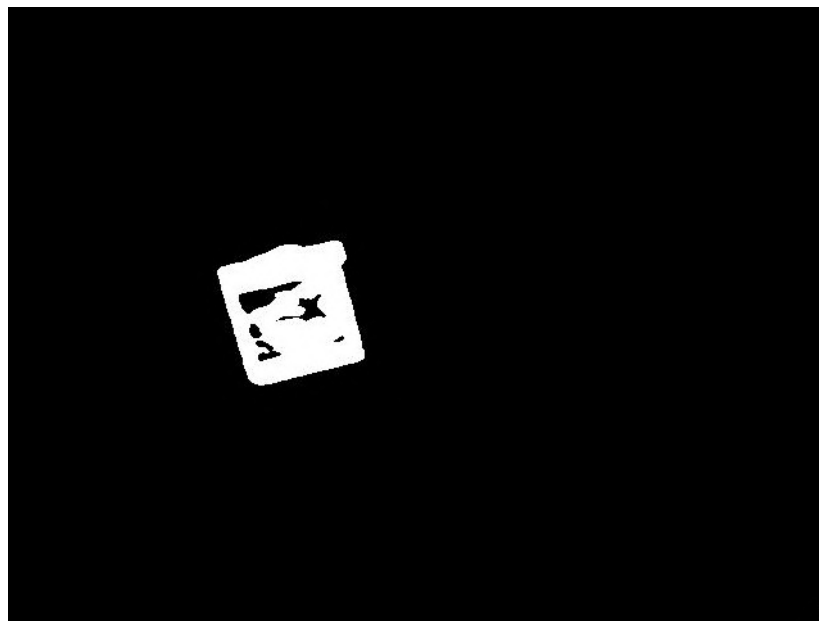


Figura 40 – Exemplo de imagem segmentada captada com fundo

figura 41.

Com intuito comparativo, foi utilizado um programa gratuito disponível na internet chamado **ImageJ**, muito utilizado para estudo principalmente em análise de processamento de imagens e uma das utilidades desse programa é a aquisição do histograma da imagem. Como pode ser observado, o programa **ImageJ** retornou um histograma bem similar ao histograma da figura em exemplo, evidenciado o comparativo na figura 42

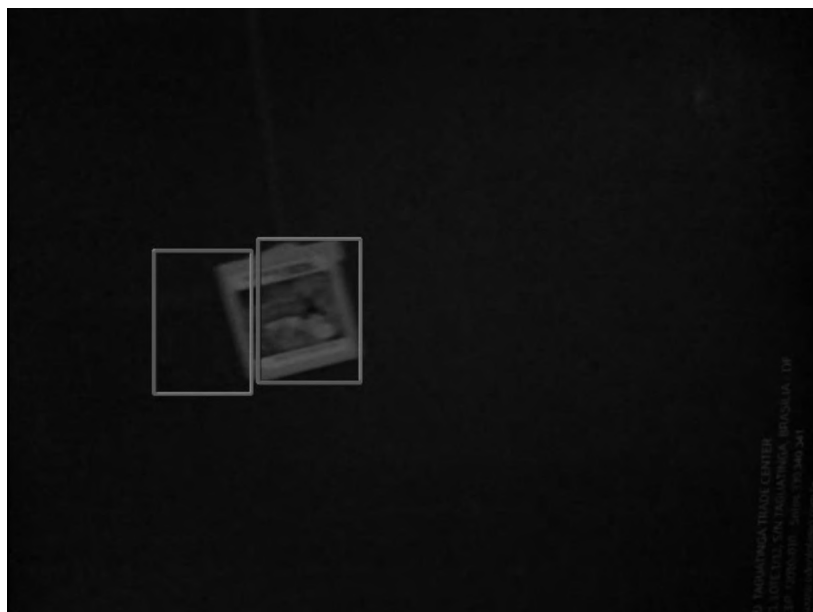


Figura 41 – Exemplo de imagem em níveis de tons de cinza

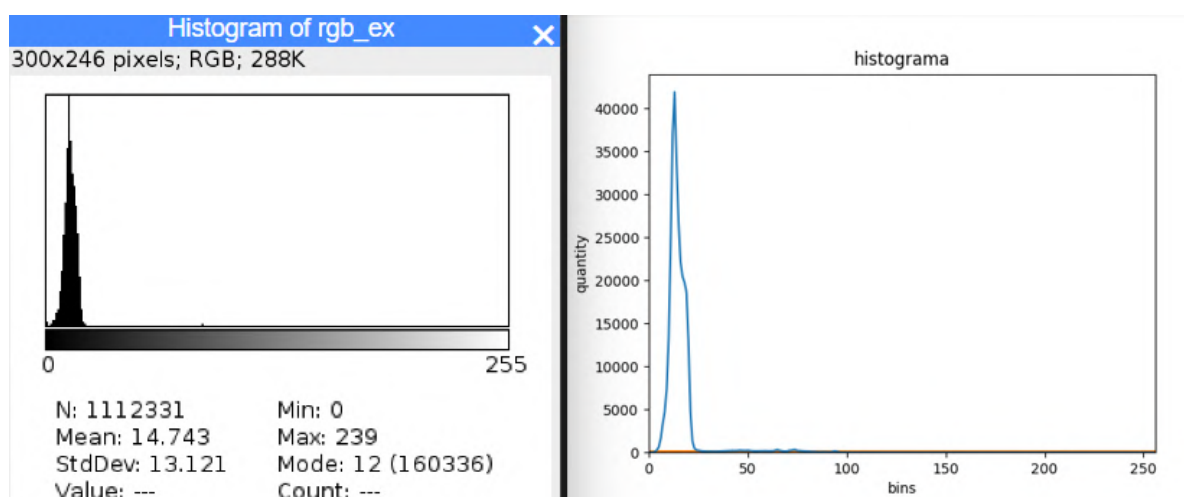


Figura 42 – Comparativo de histograma extraído do programa **ImageJ**

5 Conclusão

O principal objetivo do projeto é otimizar o armazenamento de grandes servidores com gravação de apenas objetos ou pessoas em movimento de forma rentável, visto que o custo total do projeto é o valor de uma Raspberry Pi 3B e a câmera. Outro objetivo principal do projeto teve como intuito de economizar o tempo do cliente captando apenas informações digitais de interesse para que pessoa ser analisado e identificado, de forma a captar essas imagens relevantes e as segmentá-las para uma visibilidade melhor do objeto em movimento estudado.

O objetivo de otimizar o espaço de armazenamento foi atingido a partir do captação da imagem segmentada, visto que uma imagem segmentada é 7,4% menor que uma imagem policromática, informações em mais detalhes no capítulo 4.

O método de segmentação de imagens do Otsu foi escolhido por possuir as seguintes características:

- Limiar não paramétrico e não supervisionado: trata-se de uma característica independente de fatores externos, visto que a segmentação é feita independente do grau de separabilidade em relação aos dois grupos analisados;
- Simplicidade: A técnica utilizada é bem simples e de fácil entendimento com utilização de equações de primeira ordem e sem memória;
- Visibilidade: Permite a visibilidade das características distintas de cada classe, de forma a mensurar o grau dentre elas;
- Limiar determinado e automático: Limiar estável baseado na propriedade global da imagem analisada em tempo real;

É perceptível que em luminosidade muito baixa como em 1% de 700 lumens a câmenra perde qualidade de detecção de movimento, contudo, em 33% é perceptível a mudança de eficiência, visto que a proporção do *F-measure* entre 1% e 33% é de 1,14, ou seja, uma melhora de 14%. É possível observar que a partir de 33% a eficiência se mantém constante, conclusivamente é apenas necessário um mínimo de visibilidade do objeto para que o programa seja eficiente suficientemente para uma aprovação de 90%.

O projeto trouxe essa proposta de forma a atender os objetivos gerais, concomitantemente os objetivos específicos, de tal forma que foi feito a captação e análise de imagens em movimento em tempo real, essas imagens captadas foram processadas digitalmente

pelo algoritmo com intuito de convertê-las em níveis de tons de cinza para que o histograma fosse gerado, e a partir da imagem monocromática foi convertida num processo de binarização para que finalmente, ao utilizar o método de Otsu, segmentá-las.

Assim como a conclusão levantada por [Scheidemantel \(2015\)](#), o modo *streaming* apresenta muita latência na hora de inicializar o sistema, de tal forma que objetos pequenos as quais apareçam em menos de um segundo na imagem pode ser que não seja reconhecido, contudo, o usuário pode optar em apenas deixar gravando e coletar informações assim que acessar o dispositivo. A qualidade de imagem não é interferida, portanto, ainda permanece a informação retratada.

Como comparativo, os resultados obtidos do projeto de [Bruxel \(2015\)](#) indica os mesmo resultados aqui obtidos, visto que em situações adversas e similares foi conclusivo e satisfatório conforme descrito no Capítulo 4.

Ademas, foi implementado um código onde a partir dele fosse criado um servidor via *streaming*, de forma que o cliente tenha acesso remotamente das informações em tempo real do local observado, certamente houve perda de qualidade referente a latência, mas esse tipo de problema é contornável adquirindo *hardwares* mais potentes, no quesito referente tanto ao microcomputador Raspberry Pi 3 quanto a câmera com melhor desempenho e qualidade de imagem.

5.1 Trabalhos futuros

Esse projeto teve como foco uma compreensão da utilização de captação de imagens a serem submetidas a processamento de imagens e segmentadas para uma análise mais precisa e eficiente de visão computacional por captação de movimento.

Futuramente, é possível otimizar o projeto de forma a analisar e reconhecer formas e objetos por meio da técnica de descrição de imagens, de forma a criar um banco de dados com imagens as quais desejam captar, de forma a utilizar reconhecimento de imagens por similaridade de pixels, para que possa ser reconhecida e armazenada no dispositivo para análise e processamento, tese em pauta trabalhada por [Ghellere \(2015\)](#).

Referências

- AHMED, A. A.; ECHI, M. Hawk-eye: An ai-powered threat detector for intelligent surveillance cameras. *IEEE Access*, v. 9, 2021. Disponível em: <<https://ieeexplore.ieee.org/document/9408578>>. Citado na página 36.
- BECKER, A. M. Identificação de alvo por visão computacional aplicada a um robô da competição first robotics competition. 2018. Disponível em: <<https://repositorio.ifsc.edu.br/handle/123456789/297>>. Citado 2 vezes nas páginas 39 e 41.
- BRUXEL, I. L. Sistema de segurança com detecção e acompanhamento de movimento automatizado. 2015. Disponível em: <<https://core.ac.uk/download/pdf/51328889.pdf>>. Citado 3 vezes nas páginas 22, 35 e 58.
- CLAYTON, S.; NEVES, M. Estudo e implementação de técnicas de segmentação de imagens. 2001. Disponível em: <https://www.researchgate.net/publication/239927154_ESTUDO_E_IMPLEMENTACAO_DE_TECNICAS_DE_SEGMENTACAO_DE_IMAGENS>. Citado 2 vezes nas páginas 30 e 32.
- DINIZ, J. B. Análise de algoritmos de segmentação de melanoma em imagens dermatoscópicas. 2017. Disponível em: <<http://www.bcc.ufrpe.br/br/content/analise-de-algoritmos-de-segmentacao-de-melanoma-em-imagens-dermatoscopicas>>. Citado na página 36.
- ELMASRI, R. *Sistema de Banco de Dados*. [S.l.]: Afiliada, 2008. v. 1. 19 p. Citado na página 21.
- FILHO, O. M.; NETO, H. V. *Processamento digital de imagens*. [S.l.]: brasport, 1999. v. 1. 19-22 p. Citado 2 vezes nas páginas 25 e 27.
- GHELLERE, J. S. Detecção de objetos em imagens por meio da combinação de descritores locais e classificadores. 2015. Disponível em: <https://repositorio.utfpr.edu.br/jspui/bitstream/1/12518/4/MD_COCIC_2015_1_02.pdf>. Citado na página 58.
- GRIGATI, E. A.; DIAS, F. B. C. Monitoramento de tráfego veicular terrestre usando análise de movimento em imagens sequenciais. 2015. Disponível em: <https://bdm.unb.br/bitstream/10483/11046/1/2015_EstherArraesGrigati_FelipeBragaCamargoDias.pdf>. Citado na página 37.
- GUIDE, R. P. . *Raspberry Pi 3 Product Description*. 2021. Disponível em: <www.rs-components.com/raspberrypi>. Citado na página 39.
- INTELBRAS. *Manual do usuário*. 2022. Disponível em: <<https://m.media-amazon.com/images/I/B1N75BE+fVL.pdf>>. Citado na página 40.
- KOUAO, E. *Make you own Raspberry Pi Camera Stream*. 2020. Disponível em: <<https://github.com/EbenKouao/pi-camera-stream-flask>>. Citado na página 44.
- LTD, R. P. T. *Raspberry Pi Camera Algorithm and Tuning Guide*. 2021. Disponível em: <<https://datasheets.raspberrypi.com/camera/raspberry-pi-camera-guide.pdf>>. Citado 2 vezes nas páginas 27 e 40.

- MONTEIRO, L. H. Binarização por otsu e outras técnicas usadas na detecção de placas. 2003. Disponível em: <<http://www.ic.uff.br/~aconci/OTSUeOutras.pdf>>. Citado 3 vezes nas páginas 28, 29 e 33.
- MORAES, M.; SILVA, N. da. Avaliação da sobrecarga de replicação de bancos de dados relacionais para alta disponibilidade. 2021. Disponível em: <<https://repositorio.ufu.br/handle/123456789/32237>>. Citado na página 21.
- OPENCV. *About OpenCV*. 2019. Disponível em: <<https://opencv.org/about/>>. Citado na página 41.
- OTSU, N. A threshold selection method from gray-level histograms. 1979. Disponível em: <https://engineering.purdue.edu/kak/computervision/ECE661.08/OTSU_paper.pdf>. Citado na página 31.
- PEREIRA, C. A.; OLIVEIRA, S. M. M. d. Detecção de pessoas em imagens, implementando técnicas de visão computacional em um raspberry pi. 2016. Disponível em: <<http://repositorio.utfpr.edu.br/jspui/handle/1/16776>>. Citado 2 vezes nas páginas 21 e 35.
- SAÚDE, E. S. Principais itens para relatar revisões sistemáticas e meta-análises: A recomendação prisma. 2015. Disponível em: <<https://www.prisma-statement.org>>. Citado na página 23.
- SCHEIDEMANTEL, F. L. Monitoramento de vídeo por do computador raspberry pi. 2015. Disponível em: <https://bdm.unb.br/bitstream/10483/14756/1/2015_FelipeLeiteScheidemantel_tcc.pdf>. Citado 3 vezes nas páginas 27, 42 e 58.
- SINGLA, N. Motion detection based on frame difference method. 2014. Disponível em: <<http://www.irphouse.com>>. Citado na página 35.
- SOUSA, I. M. d. Estudo e desenvolvimento de um sistema de reconhecimento de expressões faciais para controlar cadeira de rodas. 2019. Disponível em: <<https://bdm.unb.br/handle/10483/25070>>. Citado 3 vezes nas páginas 13, 26 e 41.
- YAMAMOTO, R. E. A. *Gavação de detecção de movimento via streaming*. 2022. Disponível em: <<https://youtu.be/jBdOEjlOPQc>>. Citado na página 49.
- YAMAMOTO, R. E. A. *Gravação de detecção de movimento direto da Rasp*. 2022. Disponível em: <<https://youtu.be/qWDr7I9jxLI>>. Citado na página 49.
- YAMAMOTO, R. E. A. *Gravação de detecção de movimento direto da Rasp*. 2022. Disponível em: <<https://youtu.be/wfOZnPyVBPY>>. Citado na página 63.

Apêndices

APÊNDICE A – Apresentação do TCC

Neste apêndice está informado a apresentação da monografia em formato *power point* de forma a resumir e expor o projeto de uma forma mais sucinta. O vídeo de apresentação está exposto na plataforma de vídeo *YouTube* pelo autor [Yamamoto \(2022c\)](#).



Sumário

- Introdução
- Fundamentação teórica
- Metodologia
- Resultados
- Conclusão

Introdução

- Resumo
- Definição do problema
- Objetivo
 - Objetivo específico



Fundamentação teórica

- Processamento de imagens
- Segmentação de Imagens
 - Binarização/ Método de Otsu
 - Filtro Gaussiano

$$f(x, y) = \begin{bmatrix} f(0, 0) & f(0, 1) & \dots & f(0, N-1) \\ f(1, 0) & f(1, 1) & \dots & f(1, N-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(M-1, 0) & f(M-1, 1) & \dots & f(M-1, N-1) \end{bmatrix}$$



Fundamentação teórica

- Processamento de imagens
- Detecção de movimento
 - Subtração de frames

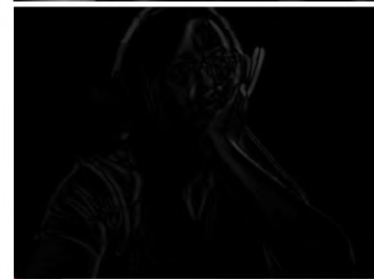


Imagem resultante subtração de frames

Fundamentação teórica

- Métrica de avaliação
- F-measure: Avalia projeto
 - Verdadeiro positivo
 - Verdadeiro negativo
 - Falso positivo
 - Falso negativo

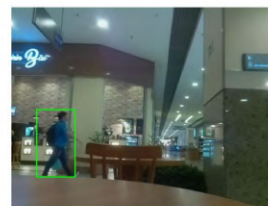


Figura 12 – Verdadeiro positivo

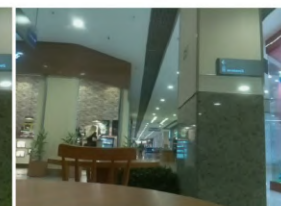


Figura 13 – Verdadeiro negativo

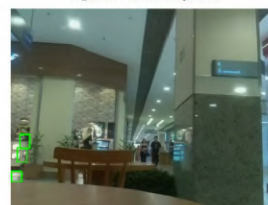


Figura 14 – Falso positivo

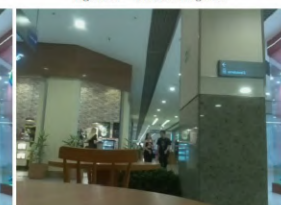


Figura 15 – Falso negativo

Fundamentação teórica

- Raspberry Pi 3 Model B
 - Afinidade: Trabalhos diante do curso
 - Custo: Uma Rasp, um periférico
 - Requisição: Microcomputador
- Camera Module V2
 - Compatibilidade
 - Custo



Fundamentação teórica



- Software
 - Linux: SO da Rasp
 - OpenCV: Proc. Img.
 - Python: Fácil Interp.
- Streaming
 - Remotamente

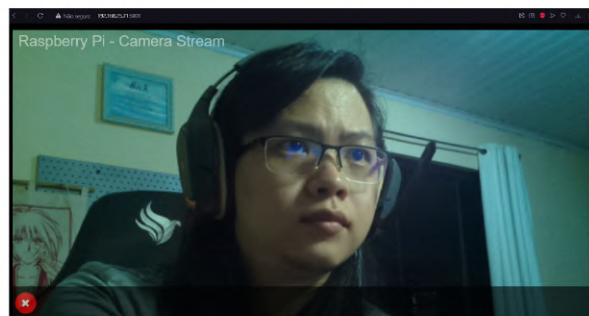


Imagem captada do navegador de internet via streaming

Metodologia

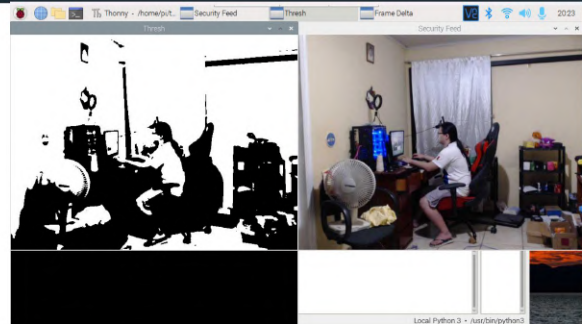
➤ Algoritmo



Imagem policromática =>

<= Variação de frames

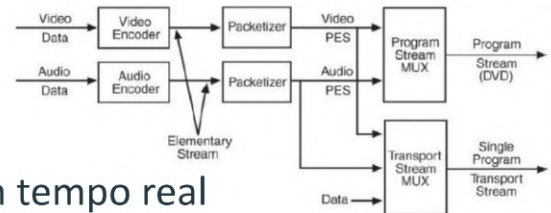
<= Imagem segmentada



```
# Interface aberta
cv2.imshow("Security Feed", frame)
cv2.imshow("Thresh", otsu)
cv2.imshow("Frame Delta", frameDelta)
```

Metodologia

➤ Captação de imagem em tempo real

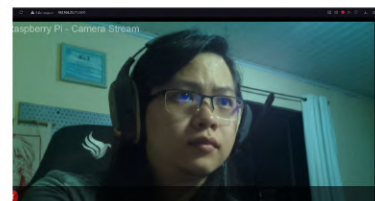


```
@app.route('/')
def index():
    return render_template('index.html') #you can customize index.html here

def gen(camera):
    #get camera frame
    while True:
        frame = camera.get_frame()
        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n\r\n')

@app.route('/video_feed')
def video_feed():
    return Response(gen(pi_camera),
                    mimetype='multipart/x-mixed-replace; boundary=frame')

if __name__ == '__main__':
    app.run(host='0.0.0.0', debug=False)
```



Metodologia

➤ Cálculo do Threshold de Otsu

```
#Função cálculo do Tresh de Otsu
def otsu_thresh(frame):
    blur = frame
    hist = cv2.calcHist([blur],[0],None,[256],[0,256])
    hist_norm = hist.ravel()/hist.sum()
    Q = hist_norm.cumsum()
    bins = np.arange(256)

    fn_min = np.inf
    thresh = -1

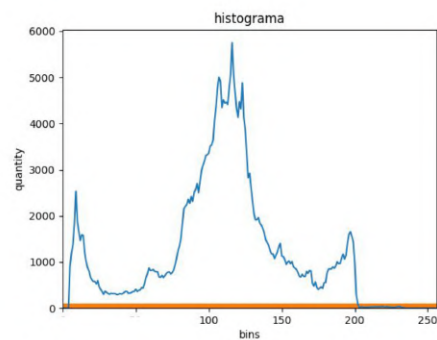
    for i in range(1,256):
        p1,p2 = np.hsplit(hist_norm,[i]) #Probabilidade de ocorrência
        q1,q2 = Q[i],Q[255]-Q[i] #Somatório de classes
        if q1 < 1.e-6 or q2 < 1.e-6:
            continue
        b1,b2 = np.hsplit(bins,[i]) # Altura
        # Cálculo da variância e desvio padrão
        m1,m2 = np.sum(p1*b1)/q1, np.sum(p2*b2)/q2
        v1,v2 = np.sum(((b1-m1)**2)*p1)/q1,np.sum(((b2-m2)**2)*p2)/q2
        # Cálculo da maximização da equação
        fn = v1*q1 + v2*q2
        if fn < fn_min:
            fn_min = fn
            thresh = i
```



Metodologia

➤ Cálculo do Histograma

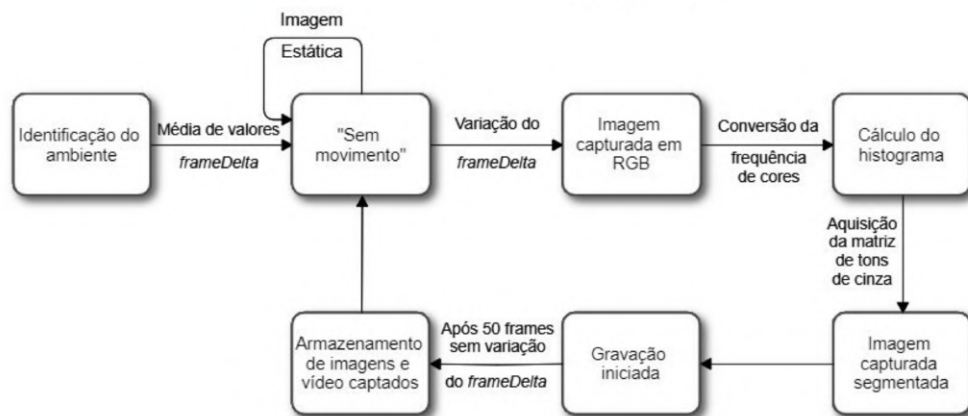
```
#Cálculo do histograma
def calculate_histogram(gray):
    dt_str = datetime.datetime.now().strftime("%Y-%m-%dT%H-%M-%S")
    hist = cv2.calcHist([gray],[0],None,[256],[0, 256])
    plt.figure()
    plt.title('histograma')
    plt.xlabel('bins')
    plt.ylabel('quantity')
    plt.plot(hist)
    plt.xlim([0, 256])
    plt.hist(hist)
    plt.savefig(f'hist_{dt_str}.png')
```



Metodologia

➤ Gravação de objetos

- frameDelta
- Contornos



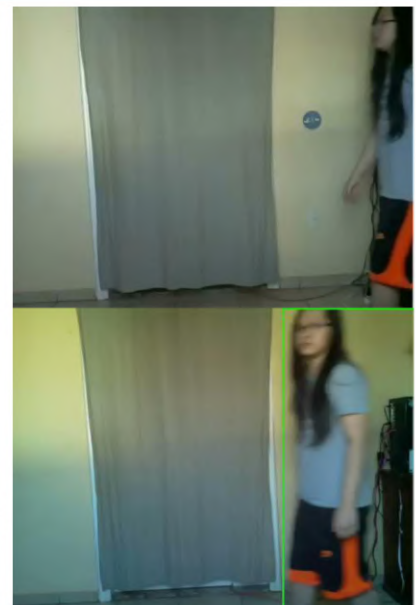
Resultados

Direto na Rasp

➤ Captação da imagem

- Resolução: 640 x 480 p
- Latência
 - 10 FPS na rasp
 - 1 FPS via streaming

Via Streaming



Resultados

- Coleta de dados
- F-Measure

$$F = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

$$\text{precision} = \frac{VP}{VP + FP}$$

$$\text{recall} = \frac{VP}{VP + FN}$$

Locais	VP	FP	FN	precision	recall	F-measure
DF Plaza	39	3	2	92,8%	95,1%	96,2%
Taguatinga Shopping	95	10	16	90,4%	85,5%	87,9%
Rua período noturno	77	14	22	84,6%	77,8%	81,1%

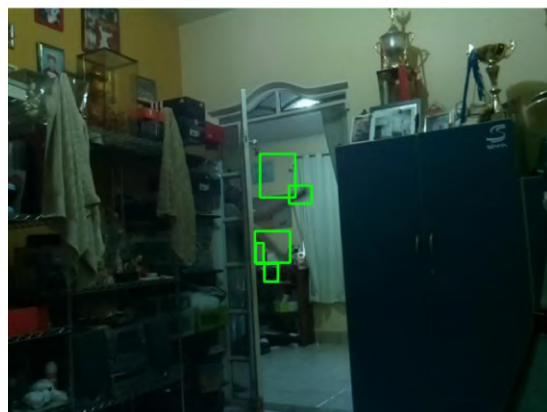
Tabela 5 – Resultados coletados de locais observados



Resultados

- Pequenos Objetos
 - 4%

$$\text{Captação} = \frac{[Altura] \times [Largura] (Imagem Movimentada)}{[Altura] \times [Largura] (Imagem Total)}$$



Contornos no tamanho de 4% da imagem total

Resultados

➤ Luminosidade variável

- Lâmpada Elgin

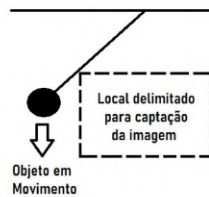


Figura 19 – Lâmpada inteligente Elgin

Níveis de luminosidade	VP	FP	FN	<i>precision</i>	<i>recall</i>	<i>F-measure</i>
1%	41	4	15	91,1%	73,2%	81,2%
33%	52	2	6	96,3%	89,6%	92,8%
66%	55	1	3	98,2%	94,8%	96,5%
100%	54	1	2	98,2%	96,4%	97,6%

Tabela 6 – Resultados coletados de luminosidade variável



Figura 35 – Imagem captada em 1% de intensidade luminosa

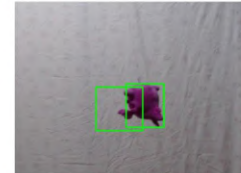


Figura 37 – Imagem captada em 66% de intensidade luminosa

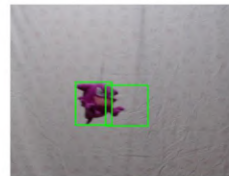


Figura 36 – Imagem captada em 33% de intensidade luminosa

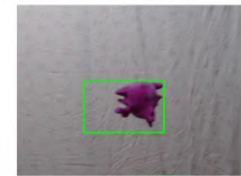
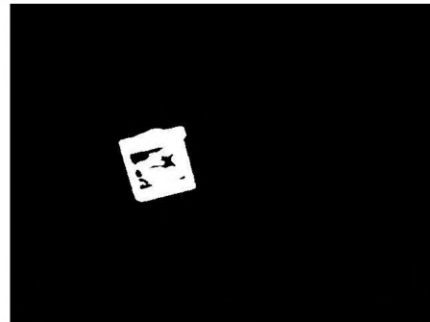


Figura 38 – Imagem captada em 100% de intensidade luminosa

Resultados

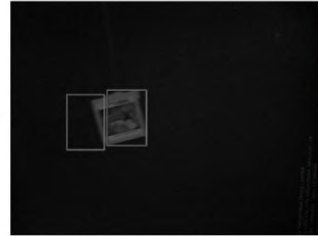
➤ Threshold de Otsu

- Alto contraste entre o objeto e o fundo

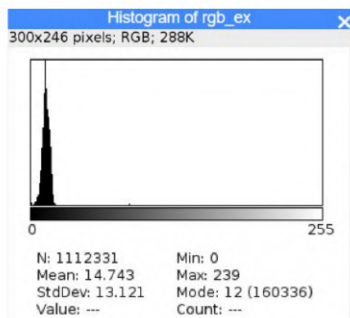


Resultados

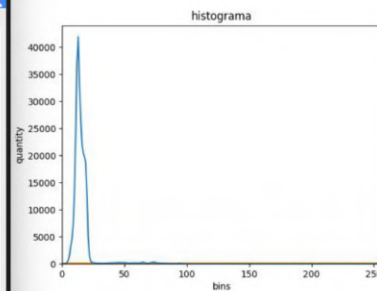
Imagem base =>



➤ Histograma gerado: comparativo



Histograma feito no ImageJ

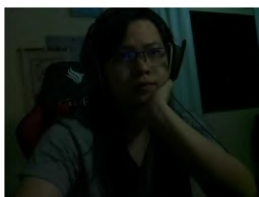


Histograma do projeto

Conclusão

➤ Utilização do método de Otsu

- Limiar não paramétrico e não supervisionado
- Simplicidade
- Visibilidade
- Limiar determinado e automático



Limiar de Otsu



Conclusão

- Otimizar o armazenamento : 7,4%
- Rentabilidade: Baixo custo de produção
- Flexibilidade: Direto ou Streaming
- Melhoria de Hardware
- Trabalhos futuros
 - Detecção de formas

Detecção de movimento por Imagem
Segmentada Utilizando Raspberry

Obrigado !

Autor: Rick Eichi Arnor Yamamoto